



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

ИТОГОВЫЙ ОТЧЕТ

по дисциплине «Разработка приложений на языке Котлин»

Студент группы ИКБО-50-23

Астахов С. П.

(подпись студента)

Принял старший преподаватель

Золотухин С. А.

(подпись руководителя)

Работа представлена

«___» _____ 2025 г.

Допущен к работе

«___» _____ 2025 г.

Москва 2025

Содержание

РТУ МИРЭА.....	1
Практическая работа 1.....	3
Практическая работа 2.....	6
Практическая работа 3.....	11
Практическая работа 4.....	17
Практическая работа 5.....	19
Практическая работа 6.....	22
Практическая работа 7.....	26
Практическая работа 8.....	29
Практическая работа 9.....	33
Практическая работа 10.....	39
Практическая работа 11.....	44
Практическая работа 12.....	51
Практическая работа 13.....	57
Практическая работа 14.....	64
Практическая работа 15.....	70
Практическая работа 16.....	72

Практическая работа 1

1. Цель работы

Изучить основные конструкции языка Котлин. Выполнить практические задания.

2. Решение

2.1. Задание 1

ДНК состоит из 4 типов нуклеотидов: А (аденин), Т (тимин), G (гуанин), С (цитозин). Ваша программа получает на вход строку вида ATGCCTCTCTC и должна посчитать число нуклеотидов каждого типа (вывести числа через пробел в порядке как в строке выше).

В ходе работы были использованы цикл, операторы, функции, коллекция, строковые шаблоны.



```
1 fun count_nycl(nycl: String) {
3     for (i in 0 until nycl.length) {
4         var nycl_1 = nycl[i]
5         map[nycl_1] = map.getOrElse(nycl_1, 0) + 1
6     }
7
8     println(
9         "A: ${map.getOrElse('A', 0)}, " +
10        "T: ${map.getOrElse('T', 0)}, " +
11        "G: ${map.getOrElse('G', 0)}, " +
12        "C: ${map.getOrElse('C', 0)}"
13    )
14 }
15
16 fun main() {
17     val nycl_str = "ATGCCTCTCTC"
18     println("Строка: $nycl_str")
19     count_nycl(nycl_str)
20 }
21
```

Код реализации алгоритма программы представлен на рисунке 1.1

Рисунок 1.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 1.2

```
~/Work/Kotlin > cd "/home/sergei/Work/Kotlin/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar  
Строка: ATGCCTCTCTC  
A: 1, T: 4, G: 1, C: 5
```

Рисунок 1.2 – Результат выполнения программы

2.2. Задание 2

В банкомате остались только купюры номиналом 8 4 2 1. Дано положительное число n - количество денег для размена. Необходимо найти минимальное количество купюр, с помощью которых можно разменять это количество денег (соблюсти порядок: первым числом вывести количество купюр номиналом 8, вторым - 4 и т д).

Для реализации задачи использовались функции, операторы, строковые шаблоны.

Код реализации алгоритма программы представлен на рисунке 1.3

```
1 fun foo(n: Int): Unit {  
2     var nom8 = n / 8  
3     var nom4 = n % 8 / 4  
4     var nom2 = n % 4 / 2  
5     var nom1 = n % 2 / 1  
6  
7     println("$n рублей | 8: $nom8, 4: $nom4, 2: $nom2, 1: $nom1")  
8 }  
9  
Run | Debug  
10 fun main() {  
11     foo(5)  
12 }  
13
```

Рисунок 1.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 1.4

```
● ~/Work/Kotlin > cd "/home/sergei/Work/Kotlin/" && kotlinc task2.kt -include-runtime -d task2.jar && j  
ava -jar task2.jar  
5 рублей| 8: 0, 4: 1, 2: 0, 1: 1
```

Рисунок 1.4 – Результат работы программы

3. Вывод

В ходе выполнения данной практической работы были изучены основные конструкции языка: var, val, fun, for, получены навыки работы с переменными, операторами, циклами, функциями, немного коллекции(словарь)

Практическая работа 2

1. Цель работы

Изучить конструкцию — список языка Котлин. Выполнить практические задания.

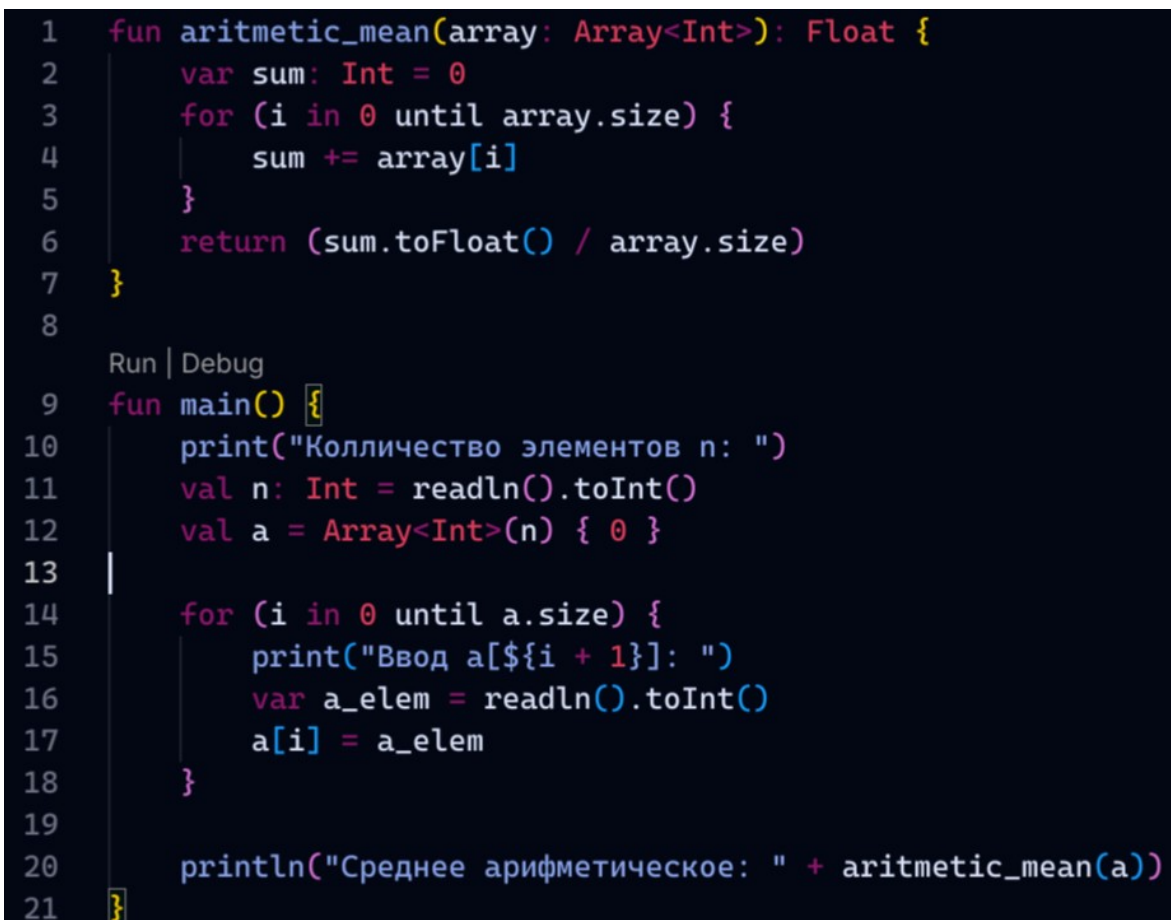
2. Решение

2.1. Задание 1

На вход подается число N — длина массива. Затем передается массив вещественных чисел (a_i) из N элементов. Необходимо подсчитать среднее арифметическое всех чисел массива. Вывести среднее арифметическое на экран.

В ходе работы были использованы список и цикл конструкции.

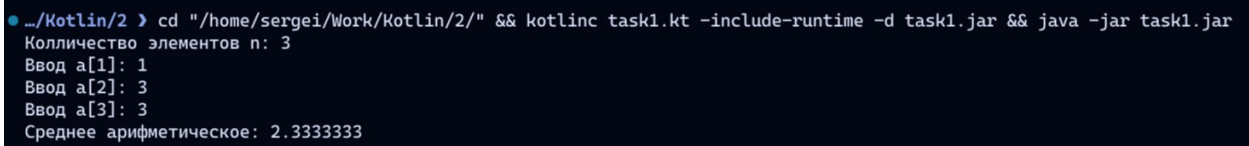
Код реализации алгоритма программы представлен на рисунке 2.1



```
1 fun arithmetic_mean(array: Array<Int>): Float {
2     var sum: Int = 0
3     for (i in 0 until array.size) {
4         sum += array[i]
5     }
6     return (sum.toFloat() / array.size)
7 }
8
9 Run | Debug
10 fun main() {
11     print("Количество элементов n: ")
12     val n: Int = readln().toInt()
13     val a = Array<Int>(n) { 0 }
14
15     for (i in 0 until a.size) {
16         print("Ввод a[${i + 1}]: ")
17         var a_elem = readln().toInt()
18         a[i] = a_elem
19     }
20
21     println("Среднее арифметическое: " + arithmetic_mean(a))
22 }
```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2



```
.../Kotlin/2 > cd "/home/sergei/Work/Kotlin/2/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Количество элементов n: 3
Ввод a[1]: 1
Ввод a[2]: 3
Ввод a[3]: 3
Среднее арифметическое: 2.3333333
```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

На вход подается число N — длина массива. Затем передается массив целых чисел ($a!$) из N элементов, отсортированный по возрастанию. Необходимо вывести на экран построчно сколько встретилось различных элементов. Каждая строка должна содержать количество элементов и сам элемент через пробел.

В ходе работы были использованы операторы, условия, циклы, функции, списки.

Код реализации алгоритма программы представлен на рисунке 2.3

```

1  fun task2(array: Array<Int>) {
2      var count = 1
3      for (i in 0 until array.size - 1) {
4          if (array[i + 1] == array[i]) {
5              count++
6          } else {
7              println("$count - ${array[i]}")
8              count = 1
9          }
10     }
11     println("$count - ${array[array.size - 1]}")
12 }
13
Run | Debug
14 fun main() {
15     print("Количество элементов n: ")
16     val n: Int = readln().toInt()
17     val a = Array<Int>(n) { 0 }
18
19     for (i in 0 until a.size) {
20         print("Ввод a[${i + 1}]: ")
21         var a_elem = readln().toInt()
22         a[i] = a_elem
23     }
24     task2(a)
25 }
26

```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

```

● ~/Kotlin/2 > cd "/home/sergei/Work/Kotlin/2/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
Количество элементов n: 5
Ввод a[1]: 2
Ввод a[2]: 3
Ввод a[3]: 4
Ввод a[4]: 4
Ввод a[5]: 5
1 - 2
1 - 3
2 - 4
1 - 5

```

Рисунок 2.4 – Результат работы программы

2.3. Задание 3

На вход подается число N — длина массива. Затем передается массив строк из N элементов (разделение через перевод строки). Каждая строка содержит только строчные символы латинского алфавита. Необходимо найти и вывести дубликат на экран. Гарантируется что он есть и только один.

В ходе работы были использованы операторы, условия, циклы, функции, список(описать что использовали).

Код реализации алгоритма программы представлен на рисунке 2.5

```

1 fun task3(array: Array<String>): String {
2     for (i in 0 until array.size) {
3         for (j in i + 1 until array.size) if (array[i].equals(array[j])) return array[i]
4     }
5     return ""
6 }
7
Run | Debug
8 fun main() {
9     print("Количество элементов n: ")
10    val n: Int = readln().toInt()
11    val a = Array<String>(n) { "" }
12
13    for (i in 0 until a.size) {
14        print("Ввод a[${i + 1}]: ")
15        var a_elem = readln()
16        a[i] = a_elem
17    }
18    println(task3(a))
19 }
20

```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```
● ~/Kotlin/2 > cd "/home/sergei/Work/Kotlin/2/" && kotlinc task3.kt -include-runtime -d task3.jar && java -jar task3.jar
Количество элементов n: 5
Ввод a[1]: zub1
Ввод a[2]: zbu
Ввод a[3]: zibb
Ввод a[4]: zubsdf
Ввод a[5]: zub1
zub1
```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе выполнения данной практической работы были изучены конструкции списка и циклов, получены навыки работы со списками.

Практическая работа 3

1. Цель работы

Изучить функции, работу со строками на языке Котлин. Выполнить практические задания.

2. Решение

2.1. Задание 1

Необходимо реализовать игру. Алгоритм игры должен быть записан в отдельной функции. В функции main должен быть только вызов функции с алгоритмом игры.

Условие следующее:

Компьютер «загадывает» (с помощью генератора случайных чисел) целое число M в промежутке от 0 до 1000 включительно. Затем предлагает пользователю угадать это число. Пользователь вводит число с клавиатуры.

Если пользователь угадал число M , то вывести на экран "Победа!". Если введенное пользователем число меньше M , то вывести на экран "Это число меньше загаданного."

Если введенное число больше, то вывести "Это число больше загаданного." Продолжать игру до тех пор, пока число не будет отгадано или пока не будет введено любое отрицательное число.

Реализовать консольное меню.

В ходе работы были использованы

Код реализации алгоритма программы представлен на рисунке 2.1

```

1  import kotlin.random.Random
2
3  fun game() {
4      val M: Int = Random.nextInt(0, 1001)
5      println("Загаданное число: " + M)
6      var choice_num = 0
7      while (M != choice_num && choice_num >= 0) {
8          print("Ввод: ")
9          choice_num = readln().toInt()
10         when (choice_num) {
11             in 0 until M -> println("Это число меньше загаданного.")
12             M -> {
13                 println("Победа!")
14                 return
15             }
16             else -> println("Это число больше загаданного.")
17         }
18     }
19 }
20
21 Run | Debug
22 fun main() {
23     game()
24 }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

● ~/Kotlin/3 > cd "/home/sergei/Work/Kotlin/3/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Загаданное число: 965
Ввод: 100
Это число меньше загаданного.
Ввод: 1000
Это число больше загаданного.
Ввод: 965
Победа!

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

На вход подается строка S, состоящая только из русских заглавных букв (без Ё). Необходимо реализовать функцию, которая кодирует переданную строку с помощью азбуки Морзе и затем вывести результат на экран. Отделять коды букв нужно пробелом. Необходимо использовать функции. В функции main должен быть вызов функции с реализацией алгоритма.

Массив с кодами Морзе:

```
{".-", "-...", ".-.", "--.", "-..", ".", "...-", "--..", "..", ".---", "-.-", ".-..", "--", "-.",  
"---", ".-.", ".-", "...", "-", ".-", "-.", "...", "-.-", "---.", "----", "-.-.", "-.-.-", "-.-.",  
"-..-", "-..-", "-.-", "-.-.-"}
```

В ходе работы были использованы when-оператор, списки, функции.

Код реализации алгоритма программы представлен на рисунке 2.3

```

1  fun convert_morze(s: String): String {
2      val morze =
3          arrayOf(
4              ".-",
5              "-.-.",
6              "--",
7              "-.-.",
8              "-.-.",
9              "--",
10             "-.-.",
11             "-.-.",
12             "-.-.",
13             "--",
14             "-.-.",
15             "-.-.",
16             "--",
17             "-.-.",
18             "-.-.",
19             "-.-.",
20             "-.-.",
21             "-.-.",
22             "-.-.",
23             "-.-.",
24             "-.-.",
25             "-.-.",
26             "-.-.",
27             "-.-.",
28             "-.-.",
29             "-.-.",
30             "-.-.",
31             "-.-.",
32             "-.-.",
33             "-.-.",
34             "-.-.",
35             "-.-.",
36         )
37     var summary_line: String = ""
38     for (element in s) {
39         summary_line += morze[element.code - 1040] + " "
40     }
41     return summary_line
42 }
43
44 fun main() {
45     println(convert_morze("АЗБУКА"))
46 }

```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

```
● ~/Work/Kotlin > cd "/home/sergei/Work/Kotlin/3/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
.- --.. -... -.- -.-
```

Рисунок 2.4 – Результат работы программы

2.3. Дополнительное задание

Создать программу, генерирующую пароль.

Необходимо использовать функции. В функции main должен быть вызов функции с реализацией алгоритма.

На вход подается число N — длина желаемого пароля. Необходимо проверить, что $N \geq 8$, иначе вывести на экран "Пароль с N количеством символов небезопасен" (подставить вместо N число) и предложить пользователю еще раз ввести число N . Если $N \geq 8$ то сгенерировать пароль, удовлетворяющий условиям ниже и вывести его на экран. В пароле должны быть:

- заглавные латинские символы
- строчные латинские символы
- числа
- специальные знаки ($_{*}$ -)

В ходе работы были использованы функции, операторы и список.

Код реализации алгоритма программы представлен на рисунке 2.5

```

1 fun generate_passowrd(n: Int): String {
2     if (n < 8) return "Пароль с $n количеством символов небезопасен"
3
4     var password: String = ""
5     var char_p =
6         arrayOf("zxcvbnmasdfghjklqwertyuiop", "ZXCVBNMASDFGHJKLQWERTYUIOP", "0123456789", "_*-")
7
8     for (i in 0 until n) {
9         var category = char_p[i % 4]
10        var randomIndex = (0 until category.length).random()
11        password += category[randomIndex]
12    }
13    return password
14 }
15
16 Run | Debug
17 fun main() {
18     var n = 0
19     while (n < 8) {
20         print("Введите длину пароля: ")
21         n = readln().toInt()
22         println(generate_passowrd(n))
23     }
24 }

```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```

• ~/Kotlin/3 > cd "/home/sergei/Work/Kotlin/3/" && kotlinc task3.kt -include-runtime -d task3.jar && java -jar task3.jar
Введите длину пароля: 2
Пароль с 2 количеством символов небезопасен
Введите длину пароля: 8
1Q4_bE0_

```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе выполнения данной практической работы была изучена работы со строками и списками, получены навыки работы с манипуляциями строк.

Практическая работа 4

1. Цель работы

Изучить алгоритм сортировки и random библиотеку

2. Решение

2.1. Задание 1

Вариант 2. Игра «Викторина»

Реализуйте консольную версию игры «Викторина». Описание:

1. Программа по очереди задает вопросы из заранее подготовленного списка.
2. Для каждого вопроса предоставляется несколько вариантов ответа.
3. Пользователь вводит номер или букву правильного ответа.
4. В конце программа показывает количество правильных ответов.

В ходе работы были использованы функции, операторы, random библиотека

Код реализации алгоритма программы представлен на рисунке 2.1

```

1 data class Question(val text: String, val options: List<String>, val correctAnswer: Int)
2
3 Run | Debug
4 fun main() {
5     val questions =
6         listOf(
7             Question(
8                 "Столица Франции?",
9                 listOf("1. Лондон", "2. Берлин", "3. Париж", "4. Мадрид"),
10                3
11            ),
12            Question(
13                "Сколько планет в Солнечной системе?",
14                listOf("1. 7", "2. 8", "3. 9", "4. 10"),
15                2
16            ),
17            Question(
18                "Автор 'Войны и мира'?",
19                listOf("1. Достоевский", "2. Толстой", "3. Пушкин", "4. Чехов"),
20                2
21            )
22        )
23     var score = 0
24
25     questions.shuffled().forEach { question ->
26         println("\n${question.text}")
27         question.options.forEach { println(it) }
28
29         print("Ваш ответ (1-4): ")
30         val answer = readLine()?.toIntOrNull()
31
32         if (answer == question.correctAnswer) {
33             println("Правильно!")
34             score++
35         } else {
36             println("Неправильно. Правильный ответ: ${question.correctAnswer}")
37         }
38     }
39     println("\nИтоговый счет: $score из ${questions.size}")
40 }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

.../Work/Kotlin > cd "/home/sergei/Work/Kotlin/4/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Автор 'Войны и мира'?
1. Достоевский
2. Толстой
3. Пушкин
4. Чехов
Ваш ответ (1-4): 1
Неправильно. Правильный ответ: 2

Столица Франции?
1. Лондон
2. Берлин
3. Париж
4. Мадрид
Ваш ответ (1-4): 3
Правильно!

Сколько планет в Солнечной системе?
1. 7
2. 8
3. 9
4. 10
Ваш ответ (1-4): 4
Неправильно. Правильный ответ: 2

Итоговый счет: 1 из 3

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Реализовать функцию, которая принимает массив `words` и целое положительное число `k`.

Необходимо вернуть `k` наиболее часто встречающихся слов.

Результирующий массив должен быть отсортирован по убыванию частоты встречаемого слова. В случае одинакового количества частоты для слов, то отсортировать и выводить их по убыванию в лексикографическом порядке.

Входные данные

Words =
["the", "day", "is", "sunny", "the", "the", "the",
"sunny", "is", "is", "day"]

Выходные данные

["the", "is", "day", "sunny"]

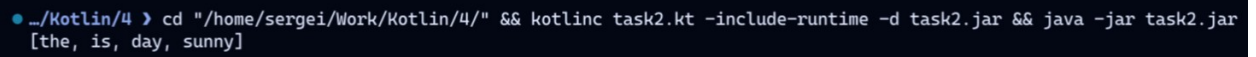
В ходе работы были использованы функции, операторы, алгоритм сортировки

Код реализации алгоритма программы представлен на рисунке 2.3

```
1 fun topKFrequent(words: Array<String>, k: Int): List<String> {
2     val frequency = mutableMapOf<String, Int>()
3
4     for (word in words) {
5         frequency[word] = frequency.getOrDefault(word, 0) + 1
6     }
7
8     return frequency
9         .entries
10        .sortedWith(compareByDescending<Map.Entry<String, Int>> { it.value }.thenBy { it.key })
11        .take(k)
12        .map { it.key }
13 }
14
15 Run | Debug
16 fun main() {
17     val words =
18         arrayOf("the", "day", "is", "sunny", "the", "the", "the", "sunny", "is", "is", "day")
19     val k = 4
20
21     val result = topKFrequent(words, k)
22     println(result)
23 }
```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

A terminal window with a dark background. The prompt is a blue dot followed by the path `~/Kotlin/4`. The command entered is `cd "/home/sergei/Work/Kotlin/4/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar`. The output is `[the, is, day, sunny]`.

```
● ~/Kotlin/4 › cd "/home/sergei/Work/Kotlin/4/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar  
[the, is, day, sunny]
```

Рисунок 2.4 – Результат работы программы

3. Вывод

В ходе выполнения данной практической работы были изучены: алгоритм сортировки, лучше поработали с random библиотекой.

Практическая работа 5

1. Цель работы

Изучить основные концепции ООП.

2. Решение

2.1. Задание 1

Создайте класс с именем Cat.

В классе должны быть следующие приватные функции:

- rest() - выводит на экран «Sleep»
- voice() - выводит на экран «Meow»
- feed() - выводит на экран «Eat»

Также необходимо реализовать одну публичную функцию:

- randomAction() - случайным образом вызывает одну из закрытых функцию.

В ходе работы были использованы классы, методы, операторы

Код реализации алгоритма программы представлен на рисунке 2.1

```

1  import kotlin.random.Random
2
3  class Cat {
4      private fun rest() = println("Sleep")
5      private fun voice() = println("Meow")
6      private fun feed() = println("Eat")
7
8      fun randomAction() {
9          when (Random.nextInt(1, 4)) {
10             1 -> rest()
11             2 -> voice()
12             3 -> feed()
13          }
14      }
15  }
16
17  Run | Debug
18  fun main() {
19      val cat = Cat()
20      repeat(5) { cat.randomAction() }
21  }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

• ~/Kotlin/4 > cd "/home/sergei/Work/Kotlin/5/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Sleep
Eat
Sleep
Eat
Eat

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Создайте класс Student.

В этом классе должны быть следующие приватные свойства:

- `var firstName: String` - имя студента. Геттер для этого свойства должен возвращать значение с первой заглавной буквой, а сеттер - убирать лишние пробелы при установке значения.
- `var lastName: String` - фамилия студента. Геттер для этого свойства также должен возвращать значение с первой заглавной буквой, а сеттер — убирать лишние пробелы при установке значения.
- `var scores: IntArray` - массив из последних десяти оценок студента

А также следующие публичные методы:

- Методы для получения и изменения свойства `firstName` с реализацией описанной логики
- Методы для получения и изменения свойства `lastName` с аналогичной логикой
- Методы для получения и изменения массива `scores`
- Метод, который добавляет новую оценку в массив `scores`, удаляя первую оценку и добавляя новую в конец (например: 2, 5, 4 → добавляем 3 → результат: 5, 4, 3). Реализовать именно через массив
- Метод, возвращающий среднюю оценку студент(считается как среднее арифметическое всех элементов массива `scores`)

В ходе работы были использованы массивы, классы, методы.

Код реализации алгоритма программы представлен на рисунке 2.3-2.4

```

1  class Student {
2      private var _firstName: String = ""
3      private var _lastName: String = ""
4      private var scores: IntArray = IntArray(10) { 0 }
5
6      var firstName: String
7          get() = _firstName.replaceFirstChar { it.uppercase() }
8          set(value) {
9              _firstName = value.trim()
10         }
11
12     var lastName: String
13         get() = _lastName.replaceFirstChar { it.uppercase() }
14         set(value) {
15             _lastName = value.trim()
16         }
17
18     fun getScores(): IntArray = scores.copyOf()
19
20     fun addScore(newScore: Int) {
21         for (i in 0 until scores.size - 1) {
22             scores[i] = scores[i + 1]
23         }
24         scores[scores.size - 1] = newScore
25     }
26
27     fun getAverageScore(): Double {
28         return if (scores.isNotEmpty()) scores.average() else 0.0
29     }
30 }
31
32 Run | Debug
33 fun main() {
34     val student = Student()
35     student.firstName = " иван "
36     student.lastName = " петров "
37
38     println("Студент: ${student.firstName} ${student.lastName}")
39
40     student.addScore(5)
41     student.addScore(4)

```

Рисунок 2.3 – Результат написания кода главной функции

```

39     student.addScore(5)
40     student.addScore(4)
41     student.addScore(3)
42
43     println("Средний балл: ${student.getAverageScore()}")
44 }
45

```

Рисунок 2.4 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.5

```
• ./Kotlin/5 > cd "/home/sergei/Work/Kotlin/5/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar  
Студент: Иван Петров  
Средний балл: 1.2
```

Рисунок 2.5 – Результат работы программы

2.3. Задание 3

Создайте класс `StudentService`.

В классе должны быть следующие публичные функции:

- **`findBestStudent()`** - принимает на вход массив объектов типа `Student` (из предыдущего задания), возвращает студента с наивысшей средней оценкой.

Если таких студентов несколько, вернуть любого.

- **`sortStudentsByLastName()`** - принимает массив объектов `Student`, сортирует его по фамилиям в алфавитном порядке и возвращает отсортированный массив

В ходе работы были использованы

Код реализации алгоритма программы представлен на рисунке 2.6

```

1  import task.Student
2
3  class StudentService {
4      fun findBestStudent(students: Array<Student>): Student? {
5          return students.maxByOrNull { it.getAverageScore() }
6      }
7
8      fun sortStudentsByLastName(students: Array<Student>): Array<Student> {
9          return students.sortedBy { it.lastName }.toTypedArray()
10     }
11 }
12
13 Run | Debug
14 fun main() {
15     val students =
16         arrayOf(
17             Student().apply {
18                 lastName = "Иванов"
19                 firstName = "Алексей"
20             },
21             Student().apply {
22                 lastName = "Петров"
23                 firstName = "Дмитрий"
24             },
25             Student().apply {
26                 lastName = "Сидоров"
27                 firstName = "Сергей"
28             }
29         )
30     val service = StudentService()
31     val sorted = service.sortStudentsByLastName(students)
32
33     println("Отсортировано по фамилии:")
34     sorted.forEach { println("${it.lastName} ${it.firstName}") }
35 }
36

```

Рисунок 2.6 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.7

```

.../Kotlin/5 > cd "/home/sergei/Work/Kotlin/5/" && kotlinc ./task2.kt ./task3.kt -include-runtime -d task3.jar && java -jar task3.jar
Отсортировано по фамилии:
Иванов Алексей
Петров Дмитрий
Сидоров Сергей

```

Рисунок 2.7 – Результат работы программы

3. Вывод

В ходе выполнения данной практической работы были изучены: основы объектно ориентированного программирования

Практическая работа 6

1. Цель работы

Изучить создание классов в Kotlin, работу с конструкторами, модификаторами доступа и свойствами.

2. Решение

2.1. Задание 1

Необходимо реализовать класс `TimeMeasure` с функционалом, описанным ниже (необходимые поля продумать самостоятельно). Обязательно должны быть реализованы валидации на входные параметры.

Конструкторы:

- Возможность создать `TimeMeasure`, задав часы, минуты и секунды.
- Возможность создать `TimeMeasure`, задав часы и минуты. Секунды тогда должны проставиться нулевыми.
- Возможность создать `TimeMeasure`, задав часы. Минуты и секунды тогда должны проставиться нулевыми.

Публичные методы:

- Вывести на экран установленное в классе время в формате `hh:mm:ss`
- Вывести на экран установленное в классе время в 12-часовом формате(используя `hh:mm:ss am/pm`)
- Метод, который прибавляет переданное время к установленному в `TimeMeasure` (на вход передаются только часы, минуты и секунды)

В ходе работы были использованы первичный/вторичный конструкторы, свойства, блок инициализации, условные операторы.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1 class TimeMeasure
2 public constructor(private var hours: Int, private var minutes: Int, private var seconds: Int) {
3     init {
4         require(hours in 0..23) { "Часы должны быть от 0 до 23" }
5         require(minutes in 0..59) { "Минуты должны быть от 0 до 59" }
6         require(seconds in 0..59) { "Секунды должны быть от 0 до 59" }
7     }
8     constructor(hours: Int, minutes: Int) : this(hours, minutes, 0)
9     constructor(hours: Int) : this(hours, 0, 0)
10    fun print24Format() {
11        println(String.format("%02d:%02d:%02d", hours, minutes, seconds))
12    }
13    fun print12Format() {
14        val period = if (hours < 12) "AM" else "PM"
15        val displayHours = if (hours == 0) 12 else if (hours > 12) hours - 12 else hours
16        println(String.format("%02d:%02d:%02d %s", displayHours, minutes, seconds, period))
17    }
18    fun addTime(addHours: Int, addMinutes: Int = 0, addSeconds: Int = 0) {
19        require(addHours >= 0 && addMinutes >= 0 && addSeconds >= 0) {
20            "Время не может быть отрицательным"
21        }
22        seconds += addSeconds
23        minutes += addMinutes + seconds / 60
24        seconds %= 60
25        hours += addHours + minutes / 60
26        minutes %= 60
27        hours %= 24
28    }
29 }
30
31 Run | Debug
32 fun main() {
33     val time1 = TimeMeasure(14, 30, 45)
34     time1.print24Format()
35     time1.print12Format()
36
37     time1.addTime(2, 15, 20)
38     println("После добавления времени:")
39     time1.print24Format()
40 }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

● ~/Kotlin/6 > cd "/home/sergei/Work/Kotlin/6/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
14:30:45
02:30:45 PM
После добавления времени:
16:46:05

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Необходимо реализовать класс `UniqueString`, который хранит внутри себя строку как массив `char` и предоставляет следующий функционал:

Конструкторы:

- Создание UniqueString, принимая на вход массив char
- Создание UniqueString, принимая на вход StringПубличные методы (названия методов, входные и выходные параметры продумать самостоятельно):
- Вернуть i-ый символ строки
- Вернуть длину строки
- Вывести строку на экран
- Проверить, есть ли переданная подстрока в UniqueString (на вход подается массив char). Вернуть true, если найдена и false иначе
- Проверить, есть ли переданная подстрока в UniqueString (на вход подается String). Вернуть true, если найдена и false иначе
- Удалить из строки UniqueString ведущие пробельные символы, если они есть
- Развернуть строку (первый символ должен стать последним, а последний первым и т.д.)

В ходе работы были использованы преобразование типов, циклы, условные операторы, работа с индексами массивов.

Код реализации алгоритма программы представлен на рисунке 2.3

```

1  class UniqueString(private var chars: CharArray) {
2      constructor(str: String) : this(str.toCharArray())
3      fun getChar(index: Int): Char {
4          require(index in chars.indices) { "Индекс вне диапазона" }
5          return chars[index]
6      }
7      fun length() = chars.size
8      fun print() {
9          println(String(chars))
10     }
11     fun contains(substring: CharArray): Boolean {
12         return String(chars).contains(String(substring))
13     }
14     fun contains(substring: String): Boolean {
15         return String(chars).contains(substring)
16     }
17     fun trimStart() {
18         var startIndex = 0
19         while (startIndex < chars.size && chars[startIndex].isWhitespace()) {
20             startIndex++
21         }
22         chars = chars.copyOfRange(startIndex, chars.size)
23     }
24     fun reverse() {
25         var left = 0
26         var right = chars.size - 1
27         while (left < right) {
28             val temp = chars[left]
29             chars[left] = chars[right]
30             chars[right] = temp
31             left++
32             right--
33         }
34     }
35 }
36
37 fun main() {
38     val str = UniqueString(" Hello World")
39     println("Длина: ${str.length()}")
40     str.trimStart()
41     str.print()
42     println("Содержит 'World': ${str.contains("World")}")
43     str.reverse()
44     println("Развернутая строка:")
45     str.print()
46 }

```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

```
● ~/Kotlin/6 > cd "/home/sergei/Work/Kotlin/6/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
Длина: 13
Hello World
Содержит 'World': true
Развернутая строка:
dlrow olleH
```

Рисунок 2.4 – Результат работы программы

2.3. Дополнительное задание

Реализовать класс «банкомат» Atm.

Создайте класс BankAccount в пакете banking

Реализуйте основной конструктор с параметрами: accountNumber, initialBalance, ownerName

Добавьте вторичный конструктор, принимающий только accountNumber и ownerName (баланс по умолчанию = 0)

Свойства и модификаторы:

- accountNumber - только для чтения (val), public
- balance - private, с кастомным геттером, возвращающим округленное значение
- ownerName - private с кастомным сеттером, проверяющим длину имени(не менее 2 символов)
- transactionCount - private, счетчик операций

Методы:

Пополнение счета (Double),

Снятие средств (проверять достаточность баланса),

Возвращение форматированной строки с информацией о счете.

Особенности:

Прямой доступ к балансу извне запрещен,

Все изменения баланса только через методы,

Необходимо добавить private метод logTransaction() для увеличения счетчика операций

Также, добавить проверку начального баланса (не может быть отрицательным) через блок инициализации.

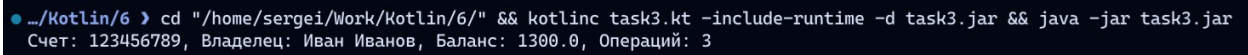
В ходе работы были использованы модификаторы доступа, кастомные аксессоры, проверки условий, приватные методы.

Код реализации алгоритма программы представлен на рисунке 2.5

```
1 package banking
2 class BankAccount(val accountNumber: String, initialBalance: Double, ownerName: String) {
3     init {
4         require(initialBalance >= 0) { "Начальный баланс не может быть отрицательным" }
5     }
6     constructor(accountNumber: String, ownerName: String) : this(accountNumber, 0.0, ownerName)
7     private var _balance: Double = 0.0
8     set(value) {
9         field = value
10        logTransaction()
11    }
12    val balance: Double
13    get() = kotlin.math.round(_balance * 100) / 100
14    private var _ownerName: String = ""
15    set(value) {
16        require(value.length >= 2) { "Имя должно содержать минимум 2 символа" }
17        field = value
18    }
19    private var transactionCount: Int = 0
20    init {
21        _balance = initialBalance
22        this._ownerName = ownerName
23    }
24    fun deposit(amount: Double) {
25        require(amount > 0) { "Сумма пополнения должна быть положительной" }
26        _balance += amount
27    }
28
29    fun withdraw(amount: Double): Boolean {
30        if (amount > 0 && _balance >= amount) {
31            _balance -= amount
32            return true
33        }
34        return false
35    }
36    private fun logTransaction() {
37        transactionCount++
38    }
39    fun getInfo(): String {
40        return "Счет: $accountNumber, Владелец: $_ownerName, Баланс: $balance, Операций: $transactionCount"
41    }
42 }
43
44 Run | Debug
45 fun main() {
46     val account = BankAccount("123456789", 1000.0, "Иван Иванов")
47     account.deposit(500.0)
48     account.withdraw(200.0)
49     println(account.getInfo())
50 }
```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

A terminal window with a dark background. The prompt is `~/Kotlin/6`. The command entered is `cd "/home/sergei/Work/Kotlin/6/" && kotlinc task3.kt -include-runtime -d task3.jar && java -jar task3.jar`. The output is `Счет: 123456789, Владелец: Иван Иванов, Баланс: 1300.0, Операций: 3`.

```
~/Kotlin/6 > cd "/home/sergei/Work/Kotlin/6/" && kotlinc task3.kt -include-runtime -d task3.jar && java -jar task3.jar
Счет: 123456789, Владелец: Иван Иванов, Баланс: 1300.0, Операций: 3
```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе работы изучены основы ООП в Kotlin: создание классов, использование конструкторов, инкапсуляция с помощью модификаторов доступа и работа со свойствами. Приобретены навыки реализации бизнес-логики в методах классов.

Практическая работа 7

1. Цель работы

Изучить принципы объектно-ориентированного программирования: наследование, абстрактные классы, enum-классы, data-классы и переопределение методов.

2. Решение

2.1. Задание

Вариант 2. Система заказов в кафе

Реализуйте иерархию классов для системы формирования заказов в кафе.

Часть 1.

1. Создайте абстрактный класс MenuItem.
2. Объявите абстрактное свойство name (название).
3. Объявите абстрактное свойство basePrice (базовая цена).
4. Объявите абстрактный метод calculateFinalPrice(), который будет вычислять итоговую цену с учетом всех надбавок (например, за размер порции).
5. Свойство id (уникальный идентификатор) должно быть задано в базовом классе как `val id: String = UUID.randomUUID().toString()`. Запретите переопределение этого свойства.

Часть 2.

1. Создайте data-класс Ingredient с полями name и isAllergen (является ли аллергеном).

Часть 3.

1. Создайте класс Drink, наследующийся от MenuItem.
2. Переопределите свойства name и basePrice.

3. Добавьте свойство `size` (тип `Size`). Создайте `enum class Size { SMALL, MEDIUM, LARGE }`.
4. Переопределите метод `calculateFinalPrice()`. Логика: `SMALL` - `basePrice * 1.0`, `MEDIUM` - `basePrice * 1.5`, `LARGE` - `basePrice * 2.0`.
5. Запретите дальнейшее переопределение метода `calculateFinalPrice()` в классах-потомках (если они будут).

Часть 4.

1. Создайте класс `Food`, наследующийся от `MenuItem`.
2. Переопределите свойства `name` и `basePrice`.
3. Добавьте свойство `ingredients: List<Ingredient>`, содержащее список ингредиентов.
4. Добавьте свойство `isVegetarian`. Реализуйте его кастомный геттер, который проверяет, что все ингредиенты не являются аллергенами (просто пример логики, в реальности это не так). Геттер должен возвращать `true`, если в списке `ingredients` нет ни одного ингредиента с `isAllergen = true`.
5. Переопределите метод `calculateFinalPrice()`. Итоговая цена равна `basePrice`.

В ходе работы были использованы абстрактные классы и методы, наследование, `data`-классы, `enum`-классы, переопределение свойств и методов, кастомные геттеры, модификатор `final`.

Код реализации алгоритма программы представлен на рисунке 2.1-2.2

```

1  import java.util.UUID
2
3  abstract class MenuItem {
4      abstract val name: String
5      abstract val basePrice: Double
6
7      abstract fun calculateFinalPrice(): Double
8
9      final val id: String = UUID.randomUUID().toString()
10 }
11
12 data class Ingredient(val name: String, val isAllergen: Boolean)
13
14 enum class Size {
15     SMALL,
16     MEDIUM,
17     LARGE
18 }
19
20 class Drink(override val name: String, override val basePrice: Double, val size: Size) :
21     MenuItem() {
22
23     final override fun calculateFinalPrice(): Double {
24         return when (size) {
25             Size.SMALL -> basePrice * 1.0
26             Size.MEDIUM -> basePrice * 1.5
27             Size.LARGE -> basePrice * 2.0
28         }
29     }
30 }
31
32 class Food(
33     override val name: String,
34     override val basePrice: Double,
35     val ingredients: List<Ingredient>
36 ) : MenuItem() {
37
38     val isVegetarian: Boolean
39     get() = ingredients.none { it.isAllergen }
40
41     override fun calculateFinalPrice(): Double {

```

Рисунок 2.1 – Результат написания кода программы

```

39         get() = ingredients.none { it.isAllergen }
40
41     override fun calculateFinalPrice(): Double {
42         return basePrice
43     }
44 }
45
46 Run | Debug
47 fun main() {
48     val drink = Drink("Кофе", 100.0, Size.LARGE)
49     println("${drink.name}: ${drink.calculateFinalPrice()}")
50
51     val food =
52         Food("Салат", 200.0, listOf(Ingredient("Помидоры", false), Ingredient("Огурцы", false)))
53     println("${food.name}: ${food.calculateFinalPrice()}, Вегетарианский: ${food.isVegetarian}")
54 }

```

Рисунок 2.2 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.3

```
.../Kotlin/5 > cd "/home/sergei/Work/Kotlin/7/" && kotlinc task.kt -include-runtime -d task.jar && java -jar task.jar  
Кофе: 200.0  
Салат: 200.0, Вегетарианский: true
```

Рисунок 2.3 – Результат выполнения программы

3. Вывод

В ходе работы изучены основы ООП: наследование, абстрактные классы, data-классы и enum-классы. Приобретены навыки проектирования иерархии классов и переопределения методов.

Практическая работа 8

1. Цель работы

Изучить работу с enum-классами в Kotlin, их свойствами и методами, а также закрепить понимание модификаторов и наследования.

2. Решение

2.1. Задание 1

Создайте enum класс для представления типов напитков. Каждый тип напитка должен иметь свойство, указывающее на его объем (например, в миллилитрах). Добавьте метод, который возвращает название напитка в формате с заглавной буквы, метод возвращения объема напитка. Добавьте метод isHot(), который возвращает true если температура > 60. Добавьте несколько напитков и выведите информацию о каждом напитке, включая его название и объем.

В ходе работы были использованы enum-классы, свойства enum-констант, методы в enum, строковые преобразования, условные операторы.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1  enum class DrinkType(
2      val name_dr: String,
3      val v: Int,
4      val temp: Int
5  ){
6      Coca_Cola("Газировка", 500, 38),
7      Tea("Чай", 250, 65),
8      Coffe("Кофе", 350, 70),
9      ColdTea("Чай", 300, 0);
10
11     fun nameDrink() = this.name_dr.replaceFirstChar {it.uppercase()}
12     fun get_v(): Int = this.v
13     fun isHot(): Boolean = temp > 60
14 }
15
16
17
18 fun main(){
19     for (dr in DrinkType.entries){
20         println("Название: ${dr.nameDrink()}")
21         println("Объем: ${dr.get_v()}")
22         if (dr.isHot())
23             | println("Напиток горячий, ${dr.temp}C")
24         else
25             | println("Напиток не горячий, ${dr.temp}C")
26         | println()
27     }
28 }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

• .../Решение/8 Main > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Котлин/Практики/Решение/8/"
  && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Название: Газировка
Объем: 500
Напиток не горячий, 38C

Название: Чай
Объем: 250
Напиток горячий, 65C

Название: Кофе
Объем: 350
Напиток горячий, 70C

Название: Чай
Объем: 300
Напиток не горячий, 0C

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

- 1) Исправьте ошибки в данном коде: <https://clck.ru/3Q7qHg>.

2) Найдите и исправьте ошибки:

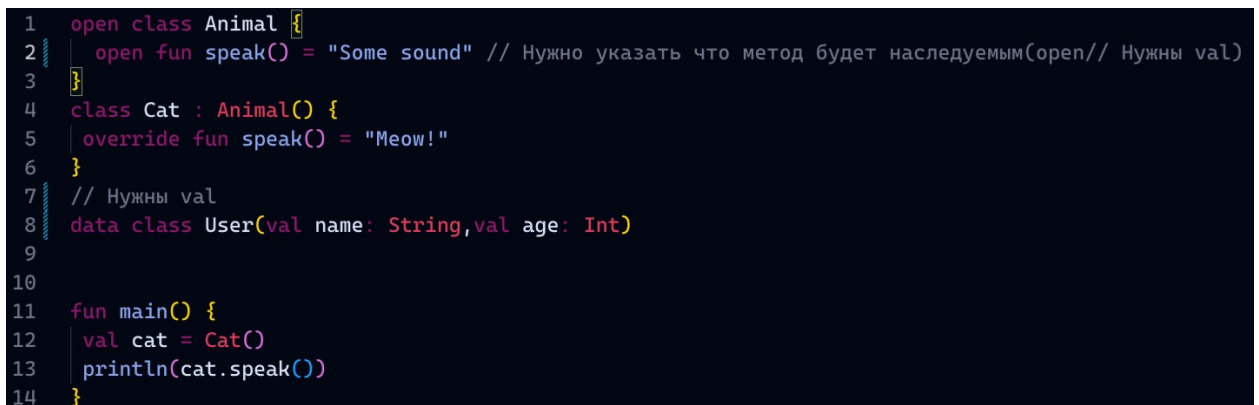
```
open class Animal {  
  
    fun speak() = "Some sound"  
  
}  
  
class Cat : Animal() {  
  
    override fun speak() = "Meow!"  
  
}  
  
fun main() {  
  
    val cat = Cat()  
  
    println(cat.speak())  
  
}
```

3) Найдите и исправьте ошибки:

```
data class User(name: String, age: Int)
```

В ходе работы были использованы модификаторы `open` и `override`, корректное объявление `data`-классов, анализ ошибок компиляции.

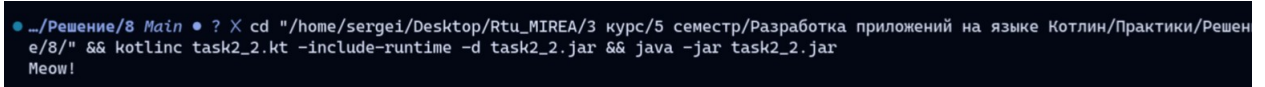
Код реализации алгоритма программы представлен на рисунке 2.3



```
1  open class Animal {  
2      open fun speak() = "Some sound" // Нужно указать что метод будет наследуемым(open// Нужны val)  
3  }  
4  class Cat : Animal() {  
5      override fun speak() = "Meow!"  
6  }  
7  // Нужны val  
8  data class User(val name: String, val age: Int)  
9  
10  
11 fun main() {  
12     val cat = Cat()  
13     println(cat.speak())  
14 }
```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4



```
~/Решение/8 Main • ? X cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решен  
e/8/" && kotlinc task2_2.kt -include-runtime -d task2_2.jar && java -jar task2_2.jar  
Meow!
```

Рисунок 2.4 – Результат работы программы

3. Вывод

В ходе работы изучены особенности епит-классов, закреплены знания по модификаторам доступа и наследованию, отработаны навыки поиска и исправления синтаксических ошибок.

Практическая работа 9

1. Цель работы

Изучить сложные аспекты ООП: интерфейсы, абстрактные классы, множественное наследование через интерфейсы и реализацию систем с состоянием.

2. Решение

2.1. Задание 1

Имеется следующий набор утверждений.

Рассматриваются следующие животные (можно заменить на другие):

- летучая мышь (Bat)
- дельфин (Dolphin)
- золотая рыбка (GoldFish)
- орел (Eagle)

Все животные одинаково едят и спят (предположим), и никто из животных не должен иметь возможности делать это иначе.

Еще животные умеют по-разному рождаться (wayOfBirth)

- Млекопитающие (Mammal) живородящие.
- Рыбы (Fish) мечут икру.
- Птицы (Bird) откладывают яйца.

Помимо этого, бывают некоторые особенности, касающиеся передвижения.

Бывают летающие животные (Flying) и плавающие (Swimming). Однако орел летает быстро, а летучая мышь медленно. Дельфин плавает быстро, а золотая рыбка медленно.

Согласно этим утверждениям, нужно создать иерархию, состоящую из классов, абстрактных классов и/или интерфейсов.

Каждое действие или утверждение подразумевает под собой вызов Unit метода, в котором реализован вывод на экран описания текущего действия. Также, необходимо реализовать систему «настройки» животных.

Каждое животное должно иметь динамические показатели:

- Голод (0-100%) - увеличивается со временем
- Энергия (0-100%) - уменьшается при активности
- Счастье (0-100%) - зависит от общего состояния

Реализовать методы для управления состоянием животных:

Базовые действия (для всех животных):

- eat() - покормить (уменьшает голод)
- sleep() - уложить спать (восстанавливает энергию)
- play() - поиграть (увеличивает счастье)

Можно, при желании, добавить особые действия.

Реализовать функциональное меню (просмотр животных, взаимодействие с каждым животным, состояние животного -> обновление состояния и т.п), протестировать созданный функционал и показать преподавателю.

В ходе работы были использованы интерфейсы, абстрактные классы, множественная реализация интерфейсов, свойства с изменяемым состоянием, методы управления состоянием, циклы и условные конструкции для меню.

Код реализации алгоритма программы представлен на рисунке 2.1-2.2

```

1  import Animal
2  import Bat
3  import Dolphin
4  import Eagle
5  import GoldFish
6
7  fun animalMenu() {
8      val animals: List<Animal> =
9          listOf(
10             Bat(),
11             Dolphin(),
12             GoldFish(),
13             Eagle(),
14         )
15
16     while (true) {
17         println("\n=== Меню управления животными ===")
18         println("1. Показать всех животных")
19         println("2. Взаимодействовать с животным")
20         println("3. Выход")
21         print("Выберите опцию: ")
22         when (readLine()) {
23             "1" -> {
24                 println("\nСписок животных:")
25                 animals.forEachIndexed { index, animal ->
26                     println("${index + 1}. ${animal.name}")
27                 }
28             }
29             "2" -> {
30                 print("Введите номер животного: ")
31                 val input = readLine()?.toIntOrNull()
32                 if (input != null && input - 1 in animals.indices) {
33                     animalActions(animals[input - 1])
34                 } else {
35                     println("Неверный номер!")
36                 }
37             }
38             "3" -> return
39             else -> println("Неверная опция!")
40         }
41     }

```

Рисунок 2.1 – Результат написания кода программы

```

40     }
41 }
42 }
43
44 fun animalActions(animal: Animal) {
45     while (true) {
46         println("\n=== Взаимодействие с ${animal.name} ===")
47         println("1. Покормить")
48         println("2. Уложить спать")
49         println("3. Играть")
50         println("4. Состояние")
51         println("5. Назад")
52         print("Выберите действие: ")
53
54         when (readLine()) {
55             "1" -> animal.eat()
56             "2" -> animal.sleep()
57             "3" -> animal.play()
58             "4" -> animal.status()
59             "5" -> return
60             else -> println("Неверное действие!")
61         }
62     }
63 }
64
65 fun main() {
66     animalMenu()
67 }

```

Рисунок 2.2 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.3

```

~/9/task1 Main • ? X cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/9/task1/" && kotlinc $(find . -name "*.kt") -include-runtime -d app.jar && java -jar app.jar

=== Меню управления животными ===
1. Показать всех животных
2. Взаимодействовать с животным
3. Выход
Выберите опцию: 1

Список животных:
1. Летучая мышь
2. Дельфин
3. Золотая рыбка
4. Орёл

=== Меню управления животными ===
1. Показать всех животных
2. Взаимодействовать с животным
3. Выход
Выберите опцию: 2
Введите номер животного: 1

=== Взаимодействие с Летучая мышь ===
1. Покормить
2. Уложить спать
3. Играть
4. Состояние
5. Назад

```

Рисунок 2.3 – Результат выполнения программы

2.2. Задание 2

Мастерская BestRepairEver занимается ремонтом различных видов электроники. Она специализируется на ремонте компьютеров и телефонов. Реализуйте метод в мастерской, который позволяет по переданному устройству определить, сможет ли мастерская его починить или нет. Метод должен возвращать результат типа `boolean`. Протестируйте метод.

В ходе работы были использованы интерфейсы или `sealed`-классы для устройств, оператор `when` для проверки типа, логические операторы.

Код реализации алгоритма программы представлен на рисунке 2.4

```

1  interface Device
2
3  class Computer : Device
4
5  class Phone : Device
6
7  class OtherDevice : Device
8
9  class BestRepairEver {
10     fun canRepair(device: Device): Boolean =
11         when (device) {
12             is Computer, is Phone -> true
13             else -> false
14         }
15     }
16
17 fun main() {
18     val repairShop = BestRepairEver()
19     val devices = listOf(Computer(), Phone(), OtherDevice())
20     devices.forEach { device ->
21         val result = repairShop.canRepair(device)
22         println("Мастерская ${if (result) "может" else "не может"} починить ${device::class.simpleName}
23         ")
24     }
25 }

```

Рисунок 2.4 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.5

```

• ~/9/task1 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/
9/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
Мастерская может починить Computer
Мастерская может починить Phone
Мастерская не может починить OtherDevice

```

Рисунок 2.5 – Результат работы программы

3. Вывод

В ходе работы изучены интерфейсы, сложные иерархии классов, реализация систем с состоянием. Приобретены навыки проектирования гибких систем с использованием полиморфизма.

Практическая работа 10

1. Цель работы

Изучить обработку исключений, создание валидаторов данных и работу с анонимными объектами в Kotlin.

2. Решение

2.1. Задание 1

Подготовка анкет для отправки на сервер встретила некоторые препятствия.

Так как фронт со своей стороны не сделал проверки, Вася оказался в трудной ситуации, и решил написать собственный класс `FormValidator` с методами для проверки входных данных перед отправкой анкеты на сервер.

Вот описание его задач:

- Метод проверяет, что длина имени составляет от 2 до 20 символов, и первая буква имени заглавная.
- Метод проверяет, что дата рождения находится в диапазоне между 01.01.1900 и текущей датой.
- Метод проверяет, что пол корректно соответствует значениям из перечисления `Gender`, которое содержит только `Male` и `Female`.
- Метод проверяет, что вес является положительным числом и может быть корректно преобразован в тип `double`.

Помогите Васе, используя классы исключений, корректно обработать анкету перед отправкой.

В ходе работы были использованы обработка исключений (`try-catch, throw`), пользовательские исключения, работа с датами (`LocalDate`), регулярные выражения, проверка условий.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1  import java.time.LocalDate
2  import java.time.format.DateTimeFormatter
3  import java.time.format.DateTimeParseException
4
5  enum class Gender {
6      MALE,
7      FEMALE,
8  }
9
10 class FormValidator {
11     fun checkName(name: String) {
12         if (name.length < 2 || name.length > 20) {
13             throw IllegalArgumentException("Длина имени должна быть от 2 до 20 символов.")
14         }
15         if (!name[0].isUpperCase()) {
16             throw IllegalArgumentException("Имя должно начинаться с заглавной буквы.")
17         }
18     }
19
20     fun checkBirthDate(birthDate: String) {
21         val formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd")
22         val date: LocalDate
23         try {
24             date = LocalDate.parse(birthDate, formatter)
25         } catch (e: DateTimeParseException) {
26             throw IllegalArgumentException("Дата должна быть в формате YYYY-MM-DD.")
27         }
28
29         val minDate = LocalDate.of(1900, 1, 1)
30         val maxDate = LocalDate.now()
31         if (date.isBefore(minDate) || date.isAfter(maxDate)) {
32             throw IllegalArgumentException("Дата рождения должна быть между 01.01.1900 и сегодняшним днём.")
33         }
34     }
35
36     fun checkGender(gender: Gender) {
37         if (gender != Gender.MALE && gender != Gender.FEMALE) {
38             throw IllegalArgumentException("Пол должен быть MALE или FEMALE.")
39         }
40     }
41 }

```

Рисунок 2.1 – Результат написания кода программы

```

40     }
41
42     fun checkWeight(weight: Double) {
43         if (weight <= 0) {
44             throw IllegalArgumentException("Вес должен быть положительным числом.")
45         }
46     }
47
48     fun validateAll(
49         name: String,
50         birthDate: String,
51         gender: Gender,
52         weight: Double,
53     ) {
54         checkName(name)
55         checkBirthDate(birthDate)
56         checkGender(gender)
57         checkWeight(weight)
58         println("Все данные корректны! Можно отправлять анкету на сервер.")
59     }
60 }
61
62 fun main() {
63     try {
64         val form = FormValidator()
65         form.validateAll(
66             name = "Василийasdasdasdasdasdadawdad",
67             birthDate = "1999-05-12",
68             gender = Gender.MALE,
69             weight = 75.5,
70         )
71     } catch (e: IllegalArgumentException) {
72         println("Ошибка проверки: ${e.message}")
73     }
74 }
75

```

Рисунок 2.2 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.3

```

.../Решение/10 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/10/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Ошибка проверки: Длина имени должна быть от 2 до 20 символов.

.../Решение/10 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/10/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Все данные корректны! Можно отправлять анкету на сервер.

```

Рисунок 2.3 – Результат выполнения программы

2.2. Задание 2

Напишите программу, которая создает анонимный объект, представляющий собой список пользователей. Каждый пользователь должен

иметь имя, возраст и список друзей. Затем программа должна использовать этот список для выполнения некоторых действий, например, для нахождения самого старшего пользователя в списке и вывода его имени и возраста.

Необходимо создать свой анонимный объект (ключевое слово `object`), который представляет собой список пользователей, и использовать его для выполнения некоторых действий.

В ходе работы были использованы анонимные объекты (`object`), коллекции (списки), функции высшего порядка (`maxBy/maxWith`), `data`-классы.

Код реализации алгоритма программы представлен на рисунке 2.3

```
1 data class User{
2     val name: String,
3     val age: Int,
4     val friends: List<String>,
5 }
6
7 fun main() {
8     val usersData =
9         object {
10             val users =
11                 listOf(
12                     User("Сергей", 20, listOf("Мария", "Дмитрий")),
13                     User("Мария", 18, listOf("Сергей", "Екатерина")),
14                     User("Дмитрий", 25, listOf("Сергей")),
15                     User("Екатерина", 28, listOf("Мария")),
16                 )
17
18             fun findOldestUser(): User? = users.maxByOrNull { it.age }
19
20             fun printAllUsers() {
21                 println("Список пользователей:")
22                 for (u in users) {
23                     println("- ${u.name}, ${u.age} лет, друзья: ${u.friends.joinToString(", ")}")
24                 }
25             }
26         }
27
28     usersData.printAllUsers()
29
30     val oldest = usersData.findOldestUser()
31     if (oldest != null) {
32         println("\nСамый старший пользователь: ${oldest.name}, возраст: ${oldest.age}")
33     } else {
34         println("\nСписок пользователей пуст!")
35     }
36 }
```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

```
~/Решение/10 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/10/" && kotlinc task2.kt -include
ntime -d task2.jar && java -jar task2.jar
Список пользователей:
- Сергей, 20 лет, друзья: Мария, Дмитрий
- Мария, 18 лет, друзья: Сергей, Екатерина
- Дмитрий, 25 лет, друзья: Сергей
- Екатерина, 28 лет, друзья: Мария

Самый старший пользователь: Екатерина, возраст: 28
```

Рисунок 2.4 – Результат работы программы

3. Вывод

В ходе работы изучены механизмы обработки исключений, создание валидаторов данных и работа с анонимными объектами. Приобретены навыки безопасной работы с пользовательским вводом и организации кода с помощью объектов.

Практическая работа 11

1. Цель работы

Изучить продвинутые возможности Kotlin: scope-функции, обобщенные типы (generics), ограничения типов и функции расширения.

2. Решение

2.1. Задание 1

Разрабатывается система управления заказами в интернет-магазине. Вам предоставлены классы Product и Order. Класс Order содержит информацию о продуктах, стоимости и статусе заказа. Ваша задача – использовать scope-функции Kotlin для более удобной работы с объектами заказа (например, инициализация заказа, обработка заказа, расчет скидки и т.п.).

В ходе работы были использованы scope-функции для инициализации, модификации и обработки объектов, цепочки вызовов, операторы безопасных вызовов.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1 package order
2 class Product (
3     val id: Int,
4     val name: String,
5     val price: Int,
6     val category: String
7 )
8
9 class Order{
10     var items: MutableList<Pair<Product, Int>> = mutableListOf()
11     var totalPrice: Double = 0.0
12     var status: String = "PENDING"
13
14     fun addProduct(product: Product, quantity: Int = 1) {
15         items.add(product to quantity)
16         totalPrice += product.price * quantity
17     }
18 }
19
20 fun main() {
21     // Создаем несколько продуктов
22     val laptop = Product(1, "Laptop", 1000, "Electronics")
23     val mouse = Product(2, "Mouse", 50, "Electronics")
24     val book = Product(3, "Kotlin Programming", 35, "Books")
25     val chair = Product(4, "Office Chair", 200, "Furniture")
26
27     val order = Order().apply {
28         addProduct(laptop)
29         addProduct(mouse, 2)
30         addProduct(book)
31         status = "PROCESSING"
32         println("Заказ создан и проинициализирован")
33     }
34     order.also {
35         println("\n--- Информация о заказе ---")
36         println("Статус: ${it.status}")
37         println("Количество товаров: ${it.items.size}")
38         println("Общая стоимость: $$${it.totalPrice}")
39     }
40
41 }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```
~/Решение/10 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/11/task1/" && kotlinc main.kt -include-runtime -d main.jar && java -jar main.jar
Заказ создан и проинициализирован

--- Информация о заказе ---
Статус: PROCESSING
Количество товаров: 3
Общая стоимость: $1135.0
```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Создайте класс `ErrorLogger<T>`, который принимает обобщенный тип `T` и содержит функцию `logError`, принимающую объект типа `T` и записывающую его в лог ошибок (для упрощения можно выводить в консоль).

- Если тип `T` – строка (`String`), функция должна добавлять к сообщению об ошибке префикс `"String error: "`.

- Если тип `T` является числом (`Number`), функция должна выводить ошибку в формате `"Numeric error: [число]"`.

- Для остальных типов `T` функция должна записывать сообщение `"Unknown error type"`.

Также создайте функцию расширения для `ErrorLogger`, которая записывает ошибку и отправляет сообщение на сервер (в данном случае просто выводит сообщение о попытке отправки).

В ходе работы были использованы обобщенные классы, проверка типа (`is`), функции расширения, перегрузка методов, вывод в консоль.

Код реализации алгоритма программы представлен на рисунке 2.3

```

1
2 class ErrorLogger<T> {
3     private val errorLog = mutableListOf<String>()
4
5
6     fun logError(error: T) {
7         val errorMessage = when (error) {
8             is String -> "String error: $error"
9             is Number -> "Numeric error: $error"
10            else -> "Unknown error type"
11        }
12        errorLog.add(errorMessage)
13        println(errorMessage)
14    }
15
16 }
17
18 fun <T> ErrorLogger<T>.logAndSend(error: T) {
19     logError(error)
20     println("Попытка отправки ошибки на сервер: $error")
21 }
22
23
24 fun main() {
25
26     val stringLogger = ErrorLogger<String>()
27     val numberLogger = ErrorLogger<Number>()
28     val anyLogger = ErrorLogger<Object>()
29
30     println("\nЛогирование строк")
31     stringLogger.logError("Файл не найден")
32     stringLogger.logError("Ошибка подключения к базе данных")
33
34
35     println("\nЛогирование чисел")
36     numberLogger.logError(404)
37     numberLogger.logError(3.14)
38     numberLogger.logError(-1)
39
40 }

```

Рисунок 2.3 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.4

```
~/11/task1 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение  
/11/task2/" && kotlinc main.kt -include-runtime -d main.jar && java -jar main.jar  
Логирование строк  
String error: Файл не найден  
String error: Ошибка подключения к базе данных  
Логирование чисел  
Numeric error: 404  
Numeric error: 3.14  
Numeric error: -1
```

Рисунок 2.4 – Результат работы программы

2.3. Задание 3

Напишите класс, который принимает обобщенный тип и выполняет проверку на допустимые значения. Класс должен работать с типами Int, Double и String, проверяя:

1. Для чисел – является ли значение больше нуля.
2. Для строк – длина строки должна быть больше 5 символов.
3. Для других типов – выводить ошибку.

Используйте ограничение обобщений, чтобы тип был либо числом, либо строкой.

В ходе работы были использованы ограничения обобщений (where T : Number, where T : String), условные операторы для проверки условий, обработка разных типов данных.

Код реализации алгоритма программы представлен на рисунке 2.5

```

1  class ValueValidator<T : Any> {
2
3      fun validate(value: T): Boolean {
4          return when (value) {
5              is Int -> value > 0
6              is Double -> value > 0
7              is String -> value.length > 5
8              else -> false
9          }.also { isValid ->
10              val message = when (value) {
11                  is Int, is Double ->
12                      if (isValid) "Число $value валидно (больше 0)"
13                      else "Число $value невалидно (должно быть больше 0)"
14                  is String ->
15                      if (isValid) "Строка '$value' валидна (длина ${value.length} > 5)"
16                      else "Строка '$value' невалидна (длина ${value.length} <= 5)"
17                  else -> "Неподдерживаемый тип: ${value::class.simpleName}"
18              }
19              println(message)
20          }
21      }
22  }
23
24
25  fun main() {
26      val validator = ValueValidator<Any>()
27
28      validator.validate(10)
29      validator.validate(-5)
30      validator.validate(3.14)
31      validator.validate(0.0)
32
33      validator.validate("Hello World")
34      validator.validate("Hi")
35      validator.validate("Kotlin")
36
37      validator.validate(true)
38      validator.validate(listOf(1, 2, 3))
39  }

```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```

● .../Практики/Решение Main > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/11/task3/" && kotlinc main.kt -include-runtime -d main.jar && java -jar main.jar
Число 10 валидно (больше 0)
Число -5 невалидно (должно быть больше 0)
Число 3.14 валидно (больше 0)
Число 0.0 невалидно (должно быть больше 0)
Строка 'Hello World' валидна (длина 11 > 5)
Строка 'Hi' невалидна (длина 2 <= 5)
Строка 'Kotlin' валидна (длина 6 > 5)
Неподдерживаемый тип: Boolean
Неподдерживаемый тип: ArrayList

```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе работы изучены score-функции для работы с объектами, обобщенные типы с ограничениями и функции расширения. Приобретены навыки написания гибкого и типобезопасного кода.

Практическая работа 12

1. Цель работы

Изучить работу с nullable-типами, безопасными вызовами и функциями расширения для практических задач.

2. Решение

2.1. Задание 1

Разработайте класс `UserProfile`, который содержит информацию о пользователе (например, имя, возраст, хобби, любимый цвет, фильм, книга). Некоторые поля могут быть пустыми или `null`. Напишите метод, который обрабатывает эти значения и выводит информацию о пользователе, используя безопасное обращение к nullable типам.

1. Реализуйте класс `UserProfile` с nullable свойствами для имени, возраста и т. д.
2. Реализуйте метод `printProfileInfo`, который будет выводить информацию о пользователе, проверяя поля на `null`.
3. Добавьте проверку на `null` для полей и безопасное обращение через операторы `?.` и `?:`.

Требования:

- Используйте nullable типы.
- Обработывайте `null` значения безопасно.

В ходе работы были использованы nullable-типы, безопасные вызовы (`?.`), оператор Элвиса (`?:`), проверки на `null`, строковые шаблоны.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1  class UserProfile (
2      val name: String?,
3      val age: Int?,
4      val robbits: String?,
5      val favoriteColor: String?,
6      val favoriteMovie: String?
7  ){
8      fun printProfileInfo(){
9          println(
10             """
11             Информация о пользователе:
12             Имя: ${name ?: "не указано"},
13             Возраст: ${age ?: "не указано"},
14             Хобби: ${robbits ?: "не указано"},
15             Любимый цвет: ${favoriteColor ?: "не указано"},
16             Фильм: ${favoriteMovie ?: "не указано"}
17             """.trimIndent())
18      }
19  }
20  fun main(args: Array<String>) {
21      val user = UserProfile(
22          name = "Sergei",
23          age = null,
24          robbits = null,
25          favoriteColor = null,
26          favoriteMovie = null
27      )
28      user.printProfileInfo()
29  }

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

.../11/task3 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Котлин/Практики/Решение
/12/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Информация о пользователе:
Имя: Sergei,
Возраст: не указано,
Хобби: не указано,
Любимый цвет: не указано,
Фильм: не указано

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Напишите программу для конвертации различных единиц измерения (например: температура, расстояние, вес, время, объем, скорость, площадь, энергия, давление, угол, цифровые данные, валюта). Для этого создайте функции расширения, которые будут преобразовывать значения в другие единицы измерения.

1. Создайте функции расширения для типов, которые выполняют преобразования, например:
 - Конвертируют температуру из Цельсия в Фаренгейты.
 - Конвертируют километры в мили.
 - Конвертируют килограммы в фунты.
 - и другие.
2. Реализовать обработку возможных ошибок ввода
3. Реализуйте консольное меню для проверки функционала.

В ходе работы были использованы функции расширения для типов `Double`, `Int`, обработка исключений при вводе, циклы для меню, математические вычисления.

Код реализации алгоритма программы представлен на рисунках 2.3-2.5

```

1  import kotlin.system.exitProcess
2
3  // Расширения для конвертации температуры
4  fun Double.celsiusToFahrenheit(): Double = this * 9 / 5 + 32
5  fun Double.fahrenheitToCelsius(): Double = (this - 32) * 5 / 9
6
7  // Расширения для конвертации расстояния
8  fun Double.kilometersToMiles(): Double = this * 0.621371
9  fun Double.milesToKilometers(): Double = this / 0.621371
10
11 // Расширения для конвертации веса
12 fun Double.kilogramsToPounds(): Double = this * 2.20462
13 fun Double.poundsToKilograms(): Double = this / 2.20462
14
15 fun main() {
16     while (true) {
17         println("=== Конвертер единиц ===")
18         println("1. Температура: Цельсий → Фаренгейт")
19         println("2. Температура: Фаренгейт → Цельсий")
20         println("3. Расстояние: Километры → Мили")
21         println("4. Расстояние: Мили → Километры")
22         println("5. Вес: Килограммы → Фунты")
23         println("6. Вес: Фунты → Килограммы")
24         println("0. Выход")
25         print("Выберите опцию: ")
26
27         when (readLine()?.toIntOrNull()) {
28             1 -> convertTemperatureToFahrenheit()
29             2 -> convertTemperatureToCelsius()
30             3 -> convertKilometersToMiles()
31             4 -> convertMilesToKilometers()
32             5 -> convertKilogramsToPounds()
33             6 -> convertPoundsToKilograms()
34             0 -> exitProcess(0)
35             else -> println("Неверный ввод, попробуйте снова.\n")
36         }
37     }
38 }
39
40 fun convertTemperatureToFahrenheit() {

```

Рисунок 2.3 – Результат написания кода главной функции

```

40 fun convertTemperatureToFahrenheit() {
41     print("Введите температуру в °C: ")
42     val celsius = readLine()?.toDoubleOrNull()
43     if (celsius != null) {
44         println("$celsius °C = ${celsius.celsiusToFahrenheit()} °F\n")
45     } else {
46         println("Ошибка ввода.\n")
47     }
48 }
49
50 fun convertTemperatureToCelsius() {
51     print("Введите температуру в °F: ")
52     val fahrenheit = readLine()?.toDoubleOrNull()
53     if (fahrenheit != null) {
54         println("$fahrenheit °F = ${fahrenheit.fahrenheitToCelsius()} °C\n")
55     } else {
56         println("Ошибка ввода.\n")
57     }
58 }
59
60 fun convertKilometersToMiles() {
61     print("Введите расстояние в км: ")
62     val km = readLine()?.toDoubleOrNull()
63     if (km != null) {
64         println("$km км = ${km.kilometersToMiles()} миль\n")
65     } else {
66         println("Ошибка ввода.\n")
67     }
68 }
69
70 fun convertMilesToKilometers() {
71     print("Введите расстояние в милях: ")
72     val miles = readLine()?.toDoubleOrNull()
73     if (miles != null) {
74         println("$miles миль = ${miles.milesToKilometers()} км\n")
75     } else {
76         println("Ошибка ввода.\n")
77     }
78 }
79
80 fun convertKilogramsToPounds() {

```

Рисунок 2.4 – Результат написания кода главной функции

```
80 fun convertKilogramsToPounds() {
81     print("Введите вес в кг: ")
82     val kg = readLine()?.toDoubleOrNull()
83     if (kg != null) {
84         println("$kg кг = ${kg.kilogramsToPounds()} фунтов\n")
85     } else {
86         println("Ошибка ввода.\n")
87     }
88 }
89
90 fun convertPoundsToKilograms() {
91     print("Введите вес в фунтах: ")
92     val pounds = readLine()?.toDoubleOrNull()
93     if (pounds != null) {
94         println("$pounds фунтов = ${pounds.poundsToKilograms()} кг\n")
95     } else {
96         println("Ошибка ввода.\n")
97     }
98 }
```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```
.../Решение/12 Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Котлин/Практики/Решение/12/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
=== Конвертер единиц ===
1. Температура: Цельсий → Фаренгейт
2. Температура: Фаренгейт → Цельсий
3. Расстояние: Километры → Мили
4. Расстояние: Мили → Километры
5. Вес: Килограммы → Фунты
6. Вес: Фунты → Килограммы
0. Выход
Выберите опцию: 1
Введите температуру в °C: 100
100.0 °C = 212.0 °F
```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе работы изучены безопасные операции с nullable-типами и создание функций расширения для практических задач. Приобретены навыки обработки пользовательского ввода и организации модульного кода.

Практическая работа 13

1. Цель работы

Изучить работу с коллекциями в Kotlin: основные операции, фильтрацию, преобразование, агрегацию и группировку данных.

2. Решение

2.1. Задание 1

Создайте приложение для анализа заказов в интернет-магазине.

Каждый заказ представлен объектом с полями `orderId`, `customerId`, `products`(список названий продуктов), `totalPrice` (сумма заказа).

Создайте изменяемую коллекцию из нескольких заказов.

Реализуйте функции для:

1. Добавления нового заказа в коллекцию.
2. Удаления заказов, сумма которых меньше заданного значения.
3. Получения списка всех уникальных клиентов (по `customerId`).
4. Подсчета общего дохода магазина по всем заказам.
5. Получения топ-3 самых дорогих заказов.

В ходе работы были использованы изменяемые коллекции (`MutableList`), функции высшего порядка (`filter`, `map`, `sumOf`, `distinct`), сортировка (`sortedByDescending`), взятие элементов (`take`).

Код реализации алгоритма программы представлен на рисунках 2.1-2.2

```

1  data class Order(
2      val orderId: Int,
3      val customerId: Int,
4      val products: List<String>,
5      val totalPrice: Double
6  )
7
8  class OrderAnalyzer {
9      private val orders = mutableListOf<Order>()
10
11      // 1. Добавление нового заказа в коллекцию
12      fun addOrder(order: Order) {
13          orders.add(order)
14          println("Заказ ${order.orderId} добавлен успешно")
15      }
16
17      // 2. Удаление заказов, сумма которых меньше заданного значения
18      fun removeOrdersBelowPrice(minPrice: Double): List<Order> {
19          val removedOrders = orders.filter { it.totalPrice < minPrice }
20          orders.removeAll { it.totalPrice < minPrice }
21          println("Удалено заказов: ${removedOrders.size}")
22          return removedOrders
23      }
24
25      // 3. Получение списка всех уникальных клиентов
26      fun getUniqueCustomers(): Set<Int> {
27          return orders.map { it.customerId }.toSet()
28      }
29
30      // 4. Подсчет общего дохода магазина
31      fun getTotalRevenue(): Double {
32          return orders.sumOf { it.totalPrice }
33      }
34
35      // 5. Получение топ-3 самых дорогих заказов
36      fun getTop3MostExpensiveOrders(): List<Order> {
37          return orders.sortedByDescending { it.totalPrice }.take(3)
38      }
39
40      fun displayAllOrders() {
41          if (orders.isEmpty()) {

```

Рисунок 2.1 – Результат написания кода программы

```

40 fun displayAllOrders() {
41     if (orders.isEmpty()) {
42         println("Заказов нет")
43     } else {
44         orders.forEach { order ->
45             println("Заказ #${order.orderId}, Клиент: ${order.customerId}, " +
46                 "Товары: ${order.products}, Сумма: ${order.totalPrice} руб.")
47         }
48     }
49 }
50
51
52 fun main() {
53     val analyzer = OrderAnalyzer()
54
55     val testOrders = listOf(
56         Order(1, 101, listOf("Ноутбук", "Мышь"), 85000.0),
57         Order(2, 102, listOf("Смартфон"), 45000.0),
58         Order(3, 101, listOf("Наушники", "Чехол"), 15000.0),
59         Order(4, 103, listOf("Планшет", "Клавиатура"), 65000.0),
60         Order(5, 104, listOf("Монитор"), 30000.0)
61     )
62
63     testOrders.forEach { analyzer.addOrder(it) }
64
65     analyzer.displayAllOrders()
66
67     // 1. Добавляем новый заказ
68     println("\n1. Добавляем новый заказ:")
69     analyzer.addOrder(Order(6, 105, listOf("Принтер", "Бумага"), 25000.0))
70
71     // 2. Удаляем заказы с суммой меньше 40000
72     println("\n2. Удаляем заказы дешевле 40000:")
73     val removedOrders = analyzer.removeOrdersBelowPrice(40000.0)
74     println("Удаленные заказы: ${removedOrders.map { it.orderId }}")
75
76

```

Рисунок 2.2 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.3

```
.../Решение/13 Main • ? > java -jar ./task1.jar
Заказ 1 добавлен успешно
Заказ 2 добавлен успешно
Заказ 3 добавлен успешно
Заказ 4 добавлен успешно
Заказ 5 добавлен успешно
Заказ #1, Клиент: 101, Товары: [Ноутбук, Мышь], Сумма: 85000.0 руб.
Заказ #2, Клиент: 102, Товары: [Смартфон], Сумма: 45000.0 руб.
Заказ #3, Клиент: 101, Товары: [Наушники, Чехол], Сумма: 15000.0 руб.
Заказ #4, Клиент: 103, Товары: [Планшет, Клавиатура], Сумма: 65000.0 руб.
Заказ #5, Клиент: 104, Товары: [Монитор], Сумма: 30000.0 руб.

1. Добавляем новый заказ:
Заказ 6 добавлен успешно

2. Удаляем заказы дешевле 40000:
Удалено заказов: 3
Удаленные заказы: [3, 5, 6]
```

Рисунок 2.3 – Результат выполнения программы

2.2. Задание 2

Создайте программу для анализа данных о продажах. Каждая продажа представлена объектом с полями `date` (дата продажи), `product`, `quantity`, `pricePerUnit`.

Создайте неизменяемую коллекцию из нескольких продаж.

Реализуйте:

1. Подсчет общей выручки за все время.
2. Группировку продаж по продуктам и подсчет количества проданных единиц каждого продукта.
3. Получение списка продаж за конкретный месяц.
4. Определение продукта, который принес наибольшую выручку.

В ходе работы были использованы неизменяемые коллекции (`List`), группировка (`groupBy`), агрегация (`sumBy`, `maxBy`), фильтрация по дате, работа с ассоциативными списками.

Код реализации алгоритма программы представлен на рисунке 2.3-2.5

```

1  import java.time.LocalDate
2  data class Sale(
3      val date: LocalDate,
4      val product: String,
5      val quantity: Int,
6      val pricePerUnit: Double
7  ) {
8      val totalPrice: Double
9      |   get() = quantity * pricePerUnit
10
11  class SalesAnalyzer(private val sales: List<Sale>) {
12
13      // 1. Подсчет общей выручки за все время
14      fun getTotalRevenue(): Double {
15          |   return sales.sumOf { it.totalPrice }
16      }
17
18      // 2. Группировка продаж по продуктам и подсчет количества проданных единиц
19      fun getProductsSalesSummary(): Map<String, Int> {
20          |   return sales.groupBy { it.product }
21          |       .mapValues { (_, productSales) -> productSales.sumOf { it.quantity } }
22      }
23
24      // 3. Получение списка продаж за конкретный месяц
25      fun getSalesByMonth(month: Int, year: Int): List<Sale> {
26          |   return sales.filter {
27          |       |   it.date.monthValue == month && it.date.year == year
28          |   }
29      }
30
31      // 4. Определение продукта, который принес наибольшую выручку
32      fun getTopProductByRevenue(): Pair<String, Double>? {
33          |   return sales.groupBy { it.product }
34          |       .mapValues { (_, productSales) -> productSales.sumOf { it.totalPrice } }
35          |       .maxByOrNull { it.value }
36          |       ?.let { it.key to it.value }
37      }
38
39      fun displayAllSales() {
40          |   sales.forEach { sale ->
41          |       |   println("${sale.date}: ${sale.product} x${sale.quantity} " +

```

Рисунок 2.3 – Результат написания кода главной функции

```

41         println("${sale.date}: ${sale.product} x${sale.quantity} " +
42             "no ${sale.pricePerUnit} руб. = ${sale.totalPrice} руб.")
43     }
44 }
45 }
46
47 fun main() {
48
49     val testSales = listOf(
50         Sale(LocalDate.of(2024, 1, 15), "Ноутбук", 2, 50000.0),
51         Sale(LocalDate.of(2024, 1, 20), "Смартфон", 3, 30000.0),
52         Sale(LocalDate.of(2024, 2, 5), "Ноутбук", 1, 55000.0),
53         Sale(LocalDate.of(2024, 2, 10), "Наушники", 5, 8000.0),
54         Sale(LocalDate.of(2024, 2, 25), "Смартфон", 2, 32000.0),
55         Sale(LocalDate.of(2024, 3, 1), "Планшет", 2, 25000.0),
56         Sale(LocalDate.of(2024, 3, 15), "Наушники", 3, 7500.0),
57         Sale(LocalDate.of(2024, 3, 20), "Монитор", 1, 35000.0)
58     )
59
60     val analyzer = SalesAnalyzer(testSales)
61
62     // Выводим все продажи
63     analyzer.displayAllSales()
64
65     // 1. Общая выручка
66     println("\n1. Общая выручка за все время:")
67     val totalRevenue = analyzer.getTotalRevenue()
68     println("${"%2f".format(totalRevenue)} руб.")
69
70     // 2. Группировка по продуктам
71     println("\n2. Продажи по продуктам (количество):")
72     val productsSummary = analyzer.getProductsSalesSummary()
73     productsSummary.forEach { (product, quantity) ->
74         println("$product: $quantity шт.")
75     }
76
77     // 3. Продажи за февраль 2024
78     println("\n3. Продажи за февраль 2024:")

```

Рисунок 2.4 – Результат написания кода главной функции

```

78     println("\n3. Продажи за февраль 2024:")
79     val februarySales = analyzer.getSalesByMonth(2, 2024)
80     if (februarySales.isEmpty()) {
81         println("Продаж в этом месяце нет")
82     } else {
83         februarySales.forEach { sale ->
84             println("${sale.date}: ${sale.product} x${sale.quantity} - ${sale.totalPrice} руб.")
85         }
86     }
87
88     // 4. Продукт с наибольшей выручкой
89     println("\n4. Продукт с наибольшей выручкой:")
90     val topProduct = analyzer.getTopProductByRevenue()
91     if (topProduct != null) {
92         println("${topProduct.first}: ${"%2f".format(topProduct.second)} руб.")
93     } else {
94         println("Нет данных о продажах")
95     }
96
97 }

```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```
.../Практики/Решение Main • ? > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/13/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
2024-02-25: Смартфон x2 по 32000.0 руб. = 64000.0 руб.
2024-03-01: Планшет x2 по 25000.0 руб. = 50000.0 руб.
2024-03-15: Наушники x3 по 7500.0 руб. = 22500.0 руб.
2024-03-20: Монитор x1 по 35000.0 руб. = 35000.0 руб.

1. Общая выручка за все время:
456500.00 руб.

2. Продажи по продуктам (количество):
Ноутбук: 3 шт.
Смартфон: 5 шт.
Наушники: 8 шт.
Планшет: 2 шт.
Монитор: 1 шт.

3. Продажи за февраль 2024:
2024-02-05: Ноутбук x1 - 55000.0 руб.
2024-02-10: Наушники x5 - 40000.0 руб.
2024-02-25: Смартфон x2 - 64000.0 руб.

4. Продукт с наибольшей выручкой:
Ноутбук: 155000.00 руб.
```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе работы изучены основные операции с коллекциями: фильтрация, преобразование, агрегация и группировка данных. Приобретены навыки анализа и обработки структурированных данных с использованием стандартных функций Kotlin.

Практическая работа 14

1. Цель работы

Изучить операции над множествами и сложные операции с ассоциативными списками (Map) для обработки структурированных данных.

2. Решение

2.1. Задание 1

У вас есть два списка чисел:

- listA - набор чисел, представляющий ID товаров, которые покупались за последний месяц.
- listB - набор чисел, представляющий ID товаров, которые находятся на складе.

Найдите:

1. Товары, которые покупались и находятся на складе.
2. Товары, которые покупались, но отсутствуют на складе.
3. Объедините все товары в общий список уникальных ID.

Подсказка: необходимо использовать операции со множествами.

В ходе работы были использованы множества (Set), операции intersect, subtract, union, преобразование списков в множества.

Код реализации алгоритма программы представлен на рисунке 2.1

```

1 fun main() {
2
3     val listA = listOf(101, 205, 303, 404, 505, 303, 101) // ID купленных товаров
4     val listB = listOf(303, 404, 505, 606, 707, 808) // ID товаров на складе
5
6     val setA = listA.toSet()
7     val setB = listB.toSet()
8     println("Список A: ${listA}\n" + "Список B: ${listB}\n")
9     // 1. пересечение
10    val boughtAndInStock = setA.intersect(setB).sorted()
11    println("1. Пересечение: ${boughtAndInStock}")
12
13    // 2. разность
14    val boughtButNotInStock = setA.subtract(setB).sorted()
15    println("2. Разность: ${boughtButNotInStock}")
16
17    // 3. объединение
18    val allUniqueIds = setA.union(setB).sorted()
19    println("3. Объединение: ${allUniqueIds}")
20 }
21

```

Рисунок 2.1 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.2

```

• ~/Work/Kotlin > cd "/home/sergei/Work/Kotlin/14/" && kotlinc task1.kt -include-runtime -d task1.jar && java -jar task1.jar
Список A: [101, 205, 303, 404, 505, 303, 101]
Список B: [303, 404, 505, 606, 707, 808]

1. Пересечение: [303, 404, 505]
2. Разность: [101, 205]
3. Объединение: [101, 205, 303, 404, 505, 606, 707, 808]

```

Рисунок 2.2 – Результат выполнения программы

2.2. Задание 2

Вы создаете приложение для коллекционирования и категоризации любимых мемов/цитат из фильмов или строчек из стихотворений.

Исходные данные:

Создайте список `entries`, где каждый элемент - это ассоциативный список (Map) со следующими ключами:

- «text» – строка, содержащая сам текст (например, цитату, строчку из стихотворения или подпись к мему).

- «author» – строка, содержащая автора (например, «Шекспир», «А.С.Пушкин», «Персонаж Игры Престолов», «Internet Meme»).
- «type» – строка, категория («quote», «роем», «meme»).
- «tags» – список строк, ключевые слова, описывающие запись (например, [«мотивация», «юмор», «жизнь»]).

Необходимо реализовать:

1. Фильтрацию по типу

Создайте функцию, которая возвращает все записи определенного типа (например, только цитаты или только мемы).

2. Поиск по автору

Создайте функцию, которая возвращает все записи указанного автора.

3. Поиск по тегу

Создайте функцию, которая возвращает все записи, содержащие определенный тег (например, все записи с тегом «юмор»).

4. Статистику

Получите множество всех уникальных авторов в вашей библиотеке.

Получите множество всех уникальных тегов, которые используются в библиотеке.

5. Группировку

Сгруппируйте все записи по их типу. Результатом должна быть мапа, где ключ - это тип («quote», «роем», «meme»), а значение - список всех записей этого типа.

Для демонстрации создайте консольное меню.

В ходе работы были использованы списки ассоциативных списков (List<Map>), функции filter, flatMap, groupBy, множества для уникальных значений, консольное меню с циклом while.

Код реализации алгоритма программы представлен на рисунке 2.3-2.5

```
1 // Создадим data class для типа записи
2 data class MemeEntry(
3     val text: String,
4     val author: String,
5     val type: String, // "quote", "поем", "меме"
6     val tags: List<String>
7 )
8
9 Run | Debug
10 fun main() {
11     val entries =
12         listOf(
13             MemeEntry(
14                 text = "Быть или не быть, вот в чём вопрос.",
15                 author = "Шекспир",
16                 type = "quote",
17                 tags = listOf("философия", "жизнь", "решение")
18             ),
19             MemeEntry(
20                 text = "Я помню чудное мгновенье: Передо мной явилась ты...",
21                 author = "А.С. Пушкин",
22                 type = "поем",
23                 tags = listOf("любовь", "воспоминания", "красота")
24             ),
25             MemeEntry(
26                 text = "Это фиаско, братан.",
27                 author = "Internet Меме",
28                 type = "меме",
29                 tags = listOf("юмор", "неудача", "интернет")
30             ),
31             MemeEntry(
32                 text = "Зимнее утро. Мороз и солнце; день чудесный!",
33                 author = "А.С. Пушкин",
34                 type = "поем",
35                 tags = listOf("природа", "утро", "зима", "радость")
36             ),
37             MemeEntry(
38                 text =
39                     "Когда жизнь дает тебе лимоны, не делай лимонад. Заставь жизнь забрать их обратно!",
40                 author = "Internet Меме",
41                 type = "меме",
```

Рисунок 2.3 – Результат написания кода главной функции

```

40         type = "meme",
41         tags = listOf("юмор", "мотивация", "протест")
42     )
43 }
44
45 // 1. Вывести все цитаты (quotes)
46 println("1. Все цитаты (type='quote'):")
47 entries.filter { it.type == "quote" }.forEach {
48     println("    - \"${it.text}\" (${it.author})")
49     println("        Теги: ${it.tags.joinToString()}")
50 }
51
52 // 2. Найти все записи от определенного автора
53 val authorToFind = "А.С. Пушкин"
54 println("\n2. Все записи автора '$authorToFind':")
55 entries.filter { it.author == authorToFind }.forEach {
56     println("    - \"${it.text}\" [${it.type}]")
57     println("        Теги: ${it.tags.joinToString()}")
58 }
59
60 // 3. Дополнительные операции:
61
62 // Поиск по тегу
63 val tagToFind = "юмор"
64 println("\n3. Записи с тегом '$tagToFind':")
65 entries.filter { tagToFind in it.tags }.forEach {
66     println("    - \"${it.text}\" (${it.author})")
67 }
68
69 // Группировка по типу
70 println("\n4. Группировка по типу:")
71 entries.groupBy { it.type }.forEach { (type, items) ->
72     println("    $type (${items.size}):")
73     items.forEach { println("        - ${it.text.take(30)}...") }
74 }
75
76 // Поиск всех уникальных тегов
77 val allTags = entries.flatMap { it.tags }.toSet().sorted()

```

Рисунок 2.4 – Результат написания кода главной функции

```

75
76 // Поиск всех уникальных тегов
77 val allTags = entries.flatMap { it.tags }.toSet().sorted()
78 println("\n5. Все уникальные теги: ${allTags.joinToString()}")
79
80 // Статистика по авторам
81 println("\n6. Количество записей по авторам:")
82 entries.groupingBy { it.author }.eachCount().forEach { (author, count) ->
83     println("    $author: $count записей")
84 }
85 }
86

```

Рисунок 2.5 – Результат написания кода главной функции

Результат работы программы представлен на рисунке 2.6

```

./Kotlin/14 > cd "/home/sergei/Work/Kotlin/14/" && kotlinc task2.kt -include-runtime -d task2.jar && java -jar task2.jar
1. Все цитаты (type='quote'):
  - "Быть или не быть, вот в чём вопрос." (Шекспир)
    Теги: философия, жизнь, решение

2. Все записи автора 'А.С. Пушкин':
  - "Я помню чудное мгновенье: Передо мной явилась ты..." [роет]
    Теги: любовь, воспоминания, красота
  - "Зимнее утро. Мороз и солнце; день чудесный!" [роет]
    Теги: природа, утро, зима, радость

3. Записи с тегом 'юмор':
  - "Это фиаско, братан." (Internet Meme)
  - "Когда жизнь дает тебе лимоны, не делай лимонад. Заставь жизнь забрать их обратно!" (Internet Meme)

4. Группировка по типу:
quote (1):
  - Быть или не быть, вот в чём во...
роет (2):
  - Я помню чудное мгновенье: Пере...
  - Зимнее утро. Мороз и солнце; д...
меме (2):
  - Это фиаско, братан....
  - Когда жизнь дает тебе лимоны, ...

5. Все уникальные теги: воспоминания, жизнь, зима, интернет, красота, любовь, мотивация, неудача, природа, протест, радость, решение, утро, философия
, юмор

6. Количество записей по авторам:
Шекспир: 1 записей
А.С. Пушкин: 2 записей
Internet Meme: 2 записей

```

Рисунок 2.6 – Результат работы программы

3. Вывод

В ходе работы изучены операции над множествами и сложная работа с ассоциативными списками. Приобретены навыки обработки вложенных структур данных, реализации поиска и фильтрации, а также организации интерактивного консольного приложения.

Практическая работа 15

1. Цель работы

Изучить работу с последовательностями (sequences) в Kotlin и их преимущества для цепочечных операций с большими наборами данных, а также закрепить навыки работы с функциями высшего порядка.

2. Решение

2.1. Задание

Вариант 2. Игровые достижения и статистика Создайте программу для анализа данных игроков в видеоигре. Данные:

```
data class Player(  
    val nickname: String,  
    val level: Int,  
    val score: Int,  
    val achievements: List<String>  
    val playTimeHours: Int  
)
```

Необходимо релизовать:

1. Создание последовательности из 10–15 игроков.
2. Фильтрацию по уровню – выбрать игроков уровня > 10 .
3. Проверку наличия достижения – проверить, есть ли хотя бы один игрок с достижением «Легенда».
4. Трансформацию в статистику - преобразовать каждого игрока в строку: «Игрок: [nickname], Уровень: [level], Очки: [score]».
5. Группировку игроков по диапазонам времени: менее 50 часов, от 50 до 200 часов, более 200 часов.
6. Сортировку игроков по убыванию очков.

7. Расчет средний уровень игроков в каждой группе времени.

В ходе работы были использованы последовательности (sequence), функции высшего порядка (filter, map, any, groupBy, sortByDescending, average), лямбда-выражения, условные операторы для группировки по диапазонам.

Код реализации алгоритма программы представлен на рисунке 2.1-2.3

```
1  import kotlin.random.Random
2
3  fun generatePlayers(count: Int): Sequence<Player> = sequence {
4      val nicknames =
5          listOf(
6              "ShadowHunter",
7              "IronWarrior",
8              "MagicMage",
9              "SwiftRogue",
10             "MightyTank",
11             "SniperWolf",
12             "FireDragon",
13             "IceQueen",
14             "StormRider",
15             "DarkKnight",
16             "LightBringer",
17             "CyberNinja",
18             "SpacePirate",
19             "TimeTraveler",
20             "NeoMatrix"
21         )
22         .shuffled()
23
24     val achievementsList =
25         listOf(
26             listOf("Новичок", "Победитель арены"),
27             listOf("Новичок", "Собиратель"),
28             listOf("Новичок", "Исследователь", "Мастер подземелий"),
29             listOf("Новичок", "Победитель арены", "Богач"),
30             listOf("Новичок", "Исследователь", "Легенда"),
31             listOf("Новичок", "Победитель босса"),
32             listOf("Новичок", "Собиратель", "Мастер крафта"),
33             listOf("Новичок", "Победитель арены", "Непобедимый"),
34             listOf("Новичок", "Исследователь", "Покоритель вершин"),
35             listOf("Новичок", "Легенда", "Ветеран"),
36             listOf("Новичок", "Богач", "Торговец"),
37             listOf("Новичок", "Победитель арены", "Дуэлянт"),
38             listOf("Новичок", "Собиратель", "Коллекционер"),
39             listOf("Новичок", "Легенда", "Мифический герой"),
40             listOf("Новичок", "Исследователь", "Первооткрыватель")
41         )
42
43     repeat(count) { index ->
44         yield(
```

Рисунок 2.1 – Результат написания кода программы

```

43     repeat(count) { index ->
44         yield(
45             Player(
46                 nickname = nicknames[index % nicknames.size],
47                 level = Random.nextInt(1, 30),
48                 score = Random.nextInt(100, 10000),
49                 achievements = achievementsList[index % achievementsList.size],
50                 playTimeHours = Random.nextInt(10, 500)
51             )
52         )
53     }
54 }
55
56 data class Player(
57     val nickname: String,
58     val level: Int,
59     val score: Int,
60     val achievements: List<String>,
61     val playTimeHours: Int
62 )
63
64 fun main() {
65     // 1. Создание последовательности из 12 игроков
66     val playersSequence = generatePlayers(12)
67
68     println("=== 1. Исходный список игроков ===")
69     playersSequence.toList().forEach { println(it) }
70     val players = generatePlayers(12)
71
72     // 2. Фильтрация по уровню – выбрать игроков уровня > 10
73     println("\n=== 2. Игроки с уровнем > 10 ===")
74     val highLevelPlayers = players.filter { it.level > 10 }.toList()
75     highLevelPlayers.forEach { println("${it.nickname} - уровень: ${it.level}") }
76
77     // 3. Проверка наличия достижения «Легенда»
78     println("\n=== 3. Проверка достижения 'Легенда' ===")
79     val hasLegendAchievement = players.any { it.achievements.contains("Легенда") }
80     println("Есть ли игрок с достижением 'Легенда': $hasLegendAchievement")
81
82     if (hasLegendAchievement) {
83         val legendPlayers = players.filter { it.achievements.contains("Легенда") }.toList()
84         println("Игроки с достижением 'Легенда':")
85         legendPlayers.forEach { println(it.nickname) }
86     }

```

Рисунок 2.2 – Результат написания кода программы

```

85 | | legendPlayers.forEach { println(it.nickname) }
86 | }
87 |
88 | // 4. Трансформация в статистику
89 | println("\n=== 4. Статистика игроков ===")
90 | val playerStats =
91 |     players
92 |     .map { "Игрок: ${it.nickname}, Уровень: ${it.level}, Очки: ${it.score}" }
93 |     .toList()
94 | playerStats.forEach { println(it) }
95 |
96 | // 5. Группировка игроков по диапазонам времени
97 | println("\n=== 5. Группировка по времени игры ===")
98 | val groupedByPlayTime =
99 |     players
100 |     .groupBy { player ->
101 |         when (player.playTimeHours) {
102 |             in 0..49 -> "Менее 50 часов"
103 |             in 50..200 -> "50-200 часов"
104 |             else -> "Более 200 часов"
105 |         }
106 |     }
107 |     .mapValues { (_, playersInGroup) -> playersInGroup.toList() }
108 |
109 | groupedByPlayTime.forEach { (groupName, playersInGroup) ->
110 |     println("\n$groupName (${playersInGroup.size} игроков):")
111 |     playersInGroup.forEach { println("  ${it.nickname} - ${it.playTimeHours} ч") }
112 | }
113 |
114 | // 6. Сортировка игроков по убыванию очков
115 | println("\n=== 6. Игроки, отсортированные по очкам (по убыванию) ===")
116 | val sortedByScore = players.sortedByDescending { it.score }.toList()
117 | sortedByScore.forEachIndexed { index, player ->
118 |     println("${index + 1}. ${player.nickname} - ${player.score} очков")
119 | }
120 |
121 | // 7. Расчет среднего уровня игроков в каждой группе времени
122 | println("\n=== 7. Средний уровень по группам времени ===")
123 | groupedByPlayTime.forEach { (groupName, playersInGroup) ->
124 |     val averageLevel = playersInGroup.map { it.level }.average()
125 |     println("$groupName: средний уровень = ${"%0.2f".format(averageLevel)}")
126 | }
127 |
128 | println("\n=== Дополнительная статистика ===")
129 | val statsSequence = generatePlayers(12)
130 | val totalPlayers = statsSequence.count()
131 | println("Всего игроков: $totalPlayers")
132 | val stats = generatePlayers(12).toList()
133 | println("Средний уровень всех игроков: ${"%0.2f".format(stats.map { it.level }.average())}")
134 | println("Средние очки: ${"%0.2f".format(stats.map { it.score }.average())}")
135 | println("Среднее время игры: ${"%0.2f".format(stats.map { it.playTimeHours }.average())} часов")
136 | }

```

Рисунок 2.3 – Результат написания кода программы

Результат работы программы представлен на рисунке 2.4

```

~/Решение/15 Main > cd "/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Разработка приложений на языке Kotlin/Практики/Решение/15/" && kotlin task.kt -include-runtime -d task.jar && java -jar task.jar
=== 1. Исходный список игроков ===
Player(nickname=DarkKnight, level=18, score=4648, achievements=[Новичок, Победитель арены], playTimeHours=14)
Player(nickname=SniperWolf, level=2, score=6168, achievements=[Новичок, Собиратель], playTimeHours=242)
Player(nickname=StormRider, level=13, score=7636, achievements=[Новичок, Исследователь, Мастер подземелий], playTimeHours=398)
Player(nickname=ShadowHunter, level=8, score=8168, achievements=[Новичок, Победитель арены, Богач], playTimeHours=285)
Player(nickname=SpacePirate, level=12, score=1979, achievements=[Новичок, Исследователь, Легенда], playTimeHours=354)
Player(nickname=IceQueen, level=3, score=3491, achievements=[Новичок, Победитель босса], playTimeHours=414)
Player(nickname=TimeTraveler, level=14, score=587, achievements=[Новичок, Собиратель, Мастер крафта], playTimeHours=499)
Player(nickname=SwiftRogue, level=21, score=9484, achievements=[Новичок, Победитель арены, Непобедимый], playTimeHours=494)
Player(nickname=MightyTank, level=26, score=1993, achievements=[Новичок, Исследователь, Покоритель вершин], playTimeHours=177)
Player(nickname=NeoMatrix, level=13, score=8435, achievements=[Новичок, Легенда, Ветран], playTimeHours=219)
Player(nickname=CyberNinja, level=7, score=853, achievements=[Новичок, Богач, Торговец], playTimeHours=156)
Player(nickname=LightBringer, level=3, score=9465, achievements=[Новичок, Победитель арены, Дуэлянт], playTimeHours=439)

=== 2. Игроки с уровнем > 10 ===
MagicMage - уровень: 15
ShadowHunter - уровень: 23
SniperWolf - уровень: 13
SwiftRogue - уровень: 12
CyberNinja - уровень: 19
SpacePirate - уровень: 29
IceQueen - уровень: 15
NeoMatrix - уровень: 16

=== 3. Проверка достижения 'Легенда' ===
Есть ли игрок с достижением 'Легенда': true
Игроки с достижением 'Легенда':
LightBringer
IronWarrior

=== 4. Статистика игроков ===
Игрок: ShadowHunter, Уровень: 7, Очки: 2832
Игрок: StormRider, Уровень: 19, Очки: 4966
Игрок: SniperWolf, Уровень: 11, Очки: 5978
Игрок: TimeTraveler, Уровень: 27, Очки: 6949
Игрок: LightBringer, Уровень: 13, Очки: 7939
Игрок: IceQueen, Уровень: 19, Очки: 9784
Игрок: MagicMage, Уровень: 21, Очки: 8697
Игрок: SpacePirate, Уровень: 15, Очки: 5655
Игрок: DarkKnight, Уровень: 2, Очки: 344
Игрок: FireDragon, Уровень: 2, Очки: 996
Игрок: MightyTank, Уровень: 13, Очки: 9510
Игрок: CyberNinja, Уровень: 27, Очки: 4812

=== 5. Группировка по времени игры ===
Более 280 часов (9 игроков):
NeoMatrix - 234 ч
DarkKnight - 383 ч
StormRider - 497 ч

```

Рисунок 2.4 – Результат выполнения программы

3. Вывод

В ходе работы изучены последовательности в Kotlin и их применение для эффективной обработки данных. Приобретены навыки выполнения цепочечных операций (фильтрации, трансформации, группировки, сортировки) с сохранением ленивых вычислений. Освоены методы анализа структурированных данных с использованием стандартных функций Kotlin.

Список использованных источников

1. Синтаксис языка Котлин [Электронный ресурс]. - <https://kotlinlang.ru/docs/basic-syntax.html> (Дата обращения: 10.09.2025).
2. Основные типы данных [Электронный ресурс]. - <https://kotlinlang.ru/docs/basic-types.html> (Дата обращения: 17.09.2025).
3. Функции в языке Котлин [Электронный ресурс]. - <https://kotlinlang.ru/docs/functions.html> (Дата обращения: 20.09.2025).
4. Классы и объекты [Электронный ресурс]. - <https://kotlinlang.ru/docs/classes.html> (Дата обращения: 29.09.2025).
5. Справочник. Ключевые слова и операторы [Электронный ресурс]. - <https://kotlinlang.ru/docs/keyword-reference.html> (Дата обращения: 17.10.2025).